Project-3

Code Generation Prompts Package

Document Type	Code Generation Prompt Package
Standard	IEEE 1016 SDD -> Embedded C
Target Files	main.c and main.h ONLY
Target MCU	STM32
Total Modules	53
Total Components	134
Date	March 30, 2026
Prepared By	Prompt Builder Agent (AI-Generated)

HARD RULE: All generated code must be placed exclusively in **main.c** and **main.h**. No new files. No modifications to any other existing file.

Table of Contents

Table of Contents 2

1. Master System Prompt 5

2. Sub-Prompt 1 - ControlModule 8

3. Sub-Prompt 2 - SensorModule 14

4. Sub-Prompt 3 - CommModule 16

5. Sub-Prompt 4 - ConfigModule 19

6. Sub-Prompt 5 - SafetyModule 23

7. Sub-Prompt 6 - ComplianceModule 29

8. Sub-Prompt 7 - HALModule 31

9. Sub-Prompt 8 - SignalProcessingModule 33

10. Sub-Prompt 9 - SchedulerModule 35

11. Sub-Prompt 10 - UartModule 37

12. Sub-Prompt 11 - AdcModule 40

13. Sub-Prompt 12 - SensingModule 43

14. Sub-Prompt 13 - ControlLoopModule 46

15. Sub-Prompt 14 - HardwareAbstractionLayer 49

16. Sub-Prompt 15 - VoltageMonitorModule 51

17. Sub-Prompt 16 - FaultManagementModule 54

18. Sub-Prompt 17 - DiagnosticsModule 57

19. Sub-Prompt 18 - PowerSafetyModule 59

20. Sub-Prompt 19 - UartCommModule 61

21. Sub-Prompt 20 - CommandModule 63
22. Sub-Prompt 21 - ActuatorModule 65
23. Sub-Prompt 22 - LoggingModule 67
24. Sub-Prompt 23 - StartupModule 69
25. Sub-Prompt 24 - ADCModule 71
26. Sub-Prompt 25 - BoardSupportModule 74
27. Sub-Prompt 26 - ActuatorControlModule 76
28. Sub-Prompt 27 - HAL 78
29. Sub-Prompt 28 - Driver 80
30. Sub-Prompt 29 - Middleware 82
31. Sub-Prompt 30 - Application 84
32. Sub-Prompt 31 - StateManagementModule 87
33. Sub-Prompt 32 - WatchdogModule 89
34. Sub-Prompt 33 - FaultHandlingModule 91
35. Sub-Prompt 34 - SafetyControlModule 93
36. Sub-Prompt 35 - CommsModule 95
37. Sub-Prompt 36 - FaultMonitoringModule 98
38. Sub-Prompt 37 - PowerMonitoringModule 100
39. Sub-Prompt 38 - ReliabilityModule 102
40. Sub-Prompt 39 - StateMachineModule 104
41. Sub-Prompt 40 - SensorProcessingModule 106
42. Sub-Prompt 41 - DataPersistenceModule 108
43. Sub-Prompt 42 - SystemConstraintsModule 110

44. Sub-Prompt 43 - AcquisitionModule 112
45. Sub-Prompt 44 - CommunicationModule 114
46. Sub-Prompt 45 - FaultModule 116
47. Sub-Prompt 46 - DataLayer 119
48. Sub-Prompt 47 - StateModule 121
49. Sub-Prompt 48 - InterfaceModule 124
50. Sub-Prompt 49 - ArtifactGenerationModule 126
51. Sub-Prompt 50 - DocRepositoryModule 129
52. Sub-Prompt 51 - VCSModule 131
53. Sub-Prompt 52 - ReleasePackagingModule 133
54. Sub-Prompt 53 - TestReportingModule 135

1. Master System Prompt

The master prompt is sent as the **system message** to the code generation LLM before any module sub-prompts. It establishes the role, project context, hard rules, and coding standards that apply to every generated file.

```
================================================================================
MASTER SYSTEM PROMPT - Embedded C Code Generation
Project : Project-3
Date : March 30, 2026
Standard : IEEE 1016 SDD -> Embedded C Implementation
================================================================================

ROLE
----
You are a senior Embedded C software engineer. Your task is to implement the
software design described in the Software Design Document (SDD) for the project
"Project-3" strictly following Embedded C best practices.

PROJECT CONTEXT
---------------
Total design components : 134
Total modules : 53
Component types : Service: 74, Algorithm: 14, Class: 38, Database: 4, Module: 1, Interface: 2, API
Endpoint: 1

Modules in this project:
- ControlModule (15 components)
- SensorModule (2 components)
- CommModule (4 components)
- ConfigModule (8 components)
- SafetyModule (14 components)
- ComplianceModule (1 component)
- HALModule (3 components)
- SignalProcessingModule (1 component)
- SchedulerModule (1 component)
- UartModule (5 components)
- AdcModule (3 components)
- SensingModule (3 components)
- ControlLoopModule (3 components)
- HardwareAbstractionLayer (1 component)
- VoltageMonitorModule (3 components)
- FaultManagementModule (3 components)
```

- DiagnosticsModule (1 component)
- PowerSafetyModule (1 component)
- UartCommModule (1 component)
- CommandModule (2 components)
- ActuatorModule (1 component)
- LoggingModule (1 component)
- StartupModule (2 components)
- ADCModule (4 components)
- BoardSupportModule (2 components)
- ActuatorControlModule (1 component)
- HAL (2 components)
- Driver (1 component)
- Middleware (1 component)
- Application (4 components)
- StateManagementModule (1 component)
- WatchdogModule (2 components)
- FaultHandlingModule (1 component)
- SafetyControlModule (2 components)
- CommsModule (6 components)
- FaultMonitoringModule (1 component)
- PowerMonitoringModule (1 component)
- ReliabilityModule (1 component)
- StateMachineModule (1 component)
- SensorProcessingModule (1 component)
- DataPersistenceModule (1 component)
- SystemConstraintsModule (1 component)
- AcquisitionModule (1 component)
- CommunicationModule (1 component)
- FaultModule (4 components)
- DataLayer (1 component)
- StateModule (4 components)
- InterfaceModule (1 component)
- ArtifactGenerationModule (3 components)
- DocRepositoryModule (1 component)
- VCSModule (1 component)
- ReleasePackagingModule (3 components)
- TestReportingModule (1 component)

ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global variables derived from this design MUST be placed exclusively in main.c or main.h.
- Do NOT create any new .c or .h files.

- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

EMBEDDED C CODING STANDARDS
---

1. MISRA-C 2012 compliance where applicable.
2. No dynamic memory allocation (no malloc / calloc / free).
3. All functions must have a single entry and single exit point.
4. Maximum function length: 50 lines of executable code.
5. Every function must have a Doxygen-style comment block.
6. All global variables must be declared static in main.c.
7. Use fixed-width integer types from stdint.h.
8. Use ifndef include guards in main.h.
9. All #define macros must be UPPER_SNAKE_CASE.
10. All function names must be lower_snake_case.
11. Struct and typedef names must be PascalCase (e.g. SensorData_t).
12. Error codes must use an enum, not magic numbers.
13. Hardware register accesses must use volatile pointers or CMSIS macros.
14. ISR functions must be prefixed with ISR_ and kept minimal.
15. Avoid recursion entirely.

OUTPUT FORMAT
---

1. Output ONLY valid Embedded C code.
2. Wrap main.h additions in: /* -- ComponentID: Title -- */
3. Wrap main.c additions in: /* -- ComponentID: Title -- */
4. Never output code for files other than main.c and main.h.
5. Tag every function: @trace DD-COMP-XXX SW-REQ-XXX

==========
END OF MASTER SYSTEM PROMPT
==========

2. Sub-Prompt 1 - ControlModule

Module	ControlModule
Sub-Prompt	1 of 53
Components	DD-COMP-001, DD-COMP-002, DD-COMP-016, DD-COMP-020, DD-COMP-027, DD-COMP-030, DD-COMP-031, DD-COMP-037, DD-COMP-052, DD-COMP-088, DD-COMP-089, DD-COMP-092, DD-COMP-110, DD-COMP-112, DD-COMP-115

```
================================================================================
SUB-PROMPT 1 OF 53
Module: ControlModule | Project: Project-3
Components: 15
================================================================================

TASK: Implement all design components for module "ControlModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare algorithm function prototypes.
- Declare service init and run function prototypes.
- Define module-level enums, macros, and config constants.

main.c expected:
- Implement algorithm logic with step-by-step inline comments.
- Implement service state machine or polling loop.
- Implement module init, process, and deinit functions.

COMPONENTS TO IMPLEMENT
```

```
-----------------------
+-- DD-COMP-001 ---
| Title: ControlLoopTask
| Type: Service | Pattern: Observer | Traces: SW-REQ-001
| Description: Periodic task that executes the closed-loop control algorithm. Reads latest
validated ADC feedback, compares to target position, computes control output using configured
gains, and sends command to actuator driver.
| Inputs: Validated ADC value, target position, control mode, control gains
| Outputs: Actuator drive command (e.g., PWM duty/command value)
| Dependencies: ADCValidationService, ControlConfigStore, ActuatorDriver
| -> main.c: /* -- DD-COMP-001: ControlLoopTask -- */
| -> main.h: /* -- DD-COMP-001: ControlLoopTask -- */
| -> @trace DD-COMP-001 SW-REQ-001
+--

+-- DD-COMP-002 ---
| Title: PIDControlAlgorithm
| Type: Algorithm | Pattern: N/A | Traces: SW-REQ-001
| Description: Implements closed-loop control (e.g., PID) computing control output from error
between target and feedback, with configurable gains and saturation handling.
| Inputs: Target position, feedback position, Kp/Ki/Kd, previous error, integral state
| Outputs: Control output value
| Dependencies: ControlConfigStore
| -> main.c: /* -- DD-COMP-002: PIDControlAlgorithm -- */
| -> main.h: /* -- DD-COMP-002: PIDControlAlgorithm -- */
| -> @trace DD-COMP-002 SW-REQ-001
+--

+-- DD-COMP-016 ---
| Title: Supply Voltage Assumption
| Type: Class | Pattern: Singleton | Traces: SW-REQ-010
| Description: Holds constant for 12V actuator supply and provides scaling factor used by control
output computation for duty cycle/drive limits.
| Inputs: None (compile-time constant 12.0V)
| Outputs: Scaling factor for control output duty/drive limit calculations
| Dependencies: ControlOutputCalculator
| -> main.c: /* -- DD-COMP-016: Supply Voltage Assumption -- */
| -> main.h: /* -- DD-COMP-016: Supply Voltage Assumption -- */
| -> @trace DD-COMP-016 SW-REQ-010
+--

+-- DD-COMP-020 ---
| Title: Control Output Calculator
| Type: Class | Pattern: Strategy | Traces: SW-REQ-014
```

```
| Description: Computes actuator control output each cycle based on measured position and
commanded target. Applies scaling based on 12V assumption and output limits.
| Inputs: Measured position, target position, control parameters
| Outputs: Control output command (duty cycle/drive level)
| Dependencies: SupplyVoltageAssumption, PositionScalingAlgorithm
| -> main.c: /* -- DD-COMP-020: Control Output Calculator -- */
| -> main.h: /* -- DD-COMP-020: Control Output Calculator -- */
| -> @trace DD-COMP-020 SW-REQ-014
+--

+-- DD-COMP-027 ---
| Title: ShippingPositionStopLogic
| Type: Algorithm | Pattern: Observer | Traces: SW-REQ-018
| Description: Compares each ADC sample to the configured shipping position threshold and commands
actuator stop when crossing occurs. Logic executes within the ADC sampling ISR or high-priority
task to meet 20 ms stop requirement.
| Inputs: ADC sample stream; configured threshold; current actuator state
| Outputs: Stop command to actuator driver; event flag for crossing detection
| Dependencies: ThresholdConfigService, ActuatorDriver
| -> main.c: /* -- DD-COMP-027: ShippingPositionStopLogic -- */
| -> main.h: /* -- DD-COMP-027: ShippingPositionStopLogic -- */
| -> @trace DD-COMP-027 SW-REQ-018
+--

+-- DD-COMP-030 ---
| Title: ActuatorStateMachine
| Type: Class | Pattern: State | Traces: SW-REQ-019
| Description: Manages actuator ON/OFF states and enforces transitions triggered by UART commands.
Publishes state changes to other services for acknowledgements.
| Inputs: ON/OFF command events
| Outputs: Actuator drive enable/disable signals; state change events
| Dependencies: ActuatorDriver
| -> main.c: /* -- DD-COMP-030: ActuatorStateMachine -- */
| -> main.h: /* -- DD-COMP-030: ActuatorStateMachine -- */
| -> @trace DD-COMP-030 SW-REQ-019
+--

+-- DD-COMP-031 ---
| Title: ActuatorStateMachine_OFF
| Type: Class | Pattern: State | Traces: SW-REQ-020
| Description: Implements OFF transition path with de-energize command to actuator driver within
timing constraints.
| Inputs: OFF command event
| Outputs: Actuator drive disable signal; state change event
```

```
| Dependencies: ActuatorDriver
| -> main.c: /* -- DD-COMP-031: ActuatorStateMachine_OFF -- */
| -> main.h: /* -- DD-COMP-031: ActuatorStateMachine_OFF -- */
| -> @trace DD-COMP-031 SW-REQ-020
+--

+-- DD-COMP-037 ---
| Title: Flap Control Computation
| Type: Service | Pattern: Strategy | Traces: SW-REQ-026
| Description: Computes flap actuation commands using only the latest ADC value and configured
thresholds; no other sensor inputs are consulted.
| Inputs: latest_adc_value (int), threshold_config (struct)
| Outputs: actuation_command (enum/struct)
| Dependencies: ADCSampleBuffer, ThresholdConfigStore
| -> main.c: /* -- DD-COMP-037: Flap Control Computation -- */
| -> main.h: /* -- DD-COMP-037: Flap Control Computation -- */
| -> @trace DD-COMP-037 SW-REQ-026
+--

+-- DD-COMP-052 ---
| Title: ActuatorCommandGate
| Type: Class | Pattern: Guard | Traces: SW-REQ-041
| Description: Filters actuator movement commands and rejects them when the system is in
overvoltage or FAULT state. The gate checks the OvervoltageMonitor status before allowing commands
to reach the ActuatorController.
| Inputs: Actuator command object; overvoltage status flag; system state
| Outputs: Accepted command forwarded to ActuatorController or rejected status with reason
| Dependencies: OvervoltageMonitor; StateMachine; ActuatorController
| -> main.c: /* -- DD-COMP-052: ActuatorCommandGate -- */
| -> main.h: /* -- DD-COMP-052: ActuatorCommandGate -- */
| -> @trace DD-COMP-052 SW-REQ-041
+--

+-- DD-COMP-088 ---
| Title: ADCOnlyControlAlgorithm
| Type: Algorithm | Pattern: Strategy | Traces: SW-REQ-077
| Description: Computes closed-loop control outputs using only raw ADC feedback values and
internal control state. The algorithm explicitly excludes any non-ADC sensor inputs by design,
enforcing input validation at the interface boundary.
| Inputs: Raw ADC samples, internal controller state, configuration gains
| Outputs: Actuator command values, updated controller state
| Dependencies: ADCReader, ControlStateStore, ControlConfig
| -> main.c: /* -- DD-COMP-088: ADCOnlyControlAlgorithm -- */
| -> main.h: /* -- DD-COMP-088: ADCOnlyControlAlgorithm -- */
```

```
| -> @trace DD-COMP-088 SW-REQ-077
+--

+-- DD-COMP-089 ---
| Title: ControlLoopScheduler
| Type: Service | Pattern: Singleton | Traces: SW-REQ-078
| Description: Schedules and triggers execution of the control loop at a fixed period of 1.0 ms or
less. Uses a high-resolution timer or RTOS task period to ensure ≥1 kHz execution rate.
| Inputs: System clock/timer ticks, configured loop period
| Outputs: Control loop execution triggers
| Dependencies: RTOScheduler, HighResTimer
| -> main.c: /* -- DD-COMP-089: ControlLoopScheduler -- */
| -> main.h: /* -- DD-COMP-089: ControlLoopScheduler -- */
| -> @trace DD-COMP-089 SW-REQ-078
+--

+-- DD-COMP-092 ---
| Title: RawAdcDataPolicy
| Type: Class | Pattern: Facade | Traces: SW-REQ-081
| Description: Enforces use of raw ADC counts in control and decision logic by exposing only raw
ADC values to control algorithms and preventing any conversion to physical units within the
control path.
| Inputs: Raw ADC counts from AdcDriver
| Outputs: Raw ADC counts forwarded to control/decision logic
| Dependencies: AdcDriver, ControlStateMachine
| -> main.c: /* -- DD-COMP-092: RawAdcDataPolicy -- */
| -> main.h: /* -- DD-COMP-092: RawAdcDataPolicy -- */
| -> @trace DD-COMP-092 SW-REQ-081
+--

+-- DD-COMP-110 ---
| Title: ControlLoopScheduler
| Type: Service | Pattern: Observer | Traces: SW-REQ-097
| Description: Executes the periodic control loop. On each iteration it reads the latest ADC
feedback, invokes the actuator command computation, and issues updated commands to the actuator
driver.
| Inputs: Timer tick, latest ADC feedback sample
| Outputs: Actuator command updates
| Dependencies: ADCReader, CommandCalculator, ActuatorDriver
| -> main.c: /* -- DD-COMP-110: ControlLoopScheduler -- */
| -> main.h: /* -- DD-COMP-110: ControlLoopScheduler -- */
| -> @trace DD-COMP-110 SW-REQ-097
+--

+-- DD-COMP-112 ---
```

```
| Title: CommandCalculator
| Type: Algorithm | Pattern: Strategy | Traces: SW-REQ-097
| Description: Computes actuator command values based on the most recent ADC feedback and control
law parameters. Produces updated command signals on each loop iteration.
| Inputs: ADC value, control parameters, current actuator state
| Outputs: Computed actuator command
| Dependencies: ControlParamsStore
| -> main.c: /* -- DD-COMP-112: CommandCalculator -- */
| -> main.h: /* -- DD-COMP-112: CommandCalculator -- */
| -> @trace DD-COMP-112 SW-REQ-097
+--

+-- DD-COMP-115 ---
| Title: ActuatorEnableController
| Type: Class | Pattern: State | Traces: SW-REQ-099
| Description: Maintains actuator enable/disable state and applies state to the actuator driver.
Ensures ON/OFF commands are reflected in hardware control signals.
| Inputs: Enable/disable commands
| Outputs: Actuator enable control signal
| Dependencies: ActuatorDriver
| -> main.c: /* -- DD-COMP-115: ActuatorEnableController -- */
| -> main.h: /* -- DD-COMP-115: ActuatorEnableController -- */
| -> @trace DD-COMP-115 SW-REQ-099
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 1
================================================================================
```

3. Sub-Prompt 2 - SensorModule

Module	SensorModule
Sub-Prompt	2 of 53
Components	DD-COMP-003, DD-COMP-111

```
================================================================================
SUB-PROMPT 2 OF 53
Module: SensorModule | Project: Project-3
Components: 2
================================================================================

TASK: Implement all design components for module "SensorModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.
- Define module-level enums, macros, and config constants.

main.c expected:
- Implement service state machine or polling loop.
- Implement module init, process, and deinit functions.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-003 ---
| Title: ADCSampler
| Type: Service | Pattern: Factory | Traces: SW-REQ-002
```

```
| Description: Configures and triggers ADC sampling at a configurable rate between 100 Hz and 1
kHz, default 500 Hz, and publishes raw readings.
| Inputs: Sampling rate configuration, ADC channel
| Outputs: Raw ADC samples with timestamp
| Dependencies: ConfigService, ADCDriver
| -> main.c: /* -- DD-COMP-003: ADCSampler -- */
| -> main.h: /* -- DD-COMP-003: ADCSampler -- */
| -> @trace DD-COMP-003 SW-REQ-002
+--

+-- DD-COMP-111 ---
| Title: ADCReader
| Type: Class | Pattern: Singleton | Traces: SW-REQ-097
| Description: Provides the most recent ADC feedback sample from the ADC hardware abstraction and
caches it for control loop use.
| Inputs: ADC hardware data registers
| Outputs: Latest ADC value
| Dependencies: ADCHardwareHAL
| -> main.c: /* -- DD-COMP-111: ADCReader -- */
| -> main.h: /* -- DD-COMP-111: ADCReader -- */
| -> @trace DD-COMP-111 SW-REQ-097
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 2
================================================================================
```

4. Sub-Prompt 3 - CommModule

Module	CommModule
Sub-Prompt	3 of 53
Components	DD-COMP-004, DD-COMP-005, DD-COMP-006, DD-COMP-007

```
================================================================================
SUB-PROMPT 3 OF 53
Module: CommModule | Project: Project-3
Components: 4
================================================================================

TASK: Implement all design components for module "CommModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.
- Define module-level enums, macros, and config constants.

main.c expected:
- Implement service state machine or polling loop.
- Implement module init, process, and deinit functions.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-004 ---
| Title: UARTCommandHandler
| Type: Service | Pattern: Command | Traces: SW-REQ-003
```

```
| Description: Parses UART command frames to set target position and control mode, validates
inputs, and updates control state.
| Inputs: UART RX bytes/frames
| Outputs: Updated target position, control mode, command acknowledgements
| Dependencies: UARTDriver, CommandParser, ControlStateStore, UARTFaultHandler
| -> main.c: /* -- DD-COMP-004: UARTCommandHandler -- */
| -> main.h: /* -- DD-COMP-004: UARTCommandHandler -- */
| -> @trace DD-COMP-004 SW-REQ-003
+--

+-- DD-COMP-005 ---
| Title: CommandParser
| Type: Class | Pattern: Builder | Traces: SW-REQ-003
| Description: Parses incoming UART bytes into structured command messages with checksum and field
validation.
| Inputs: UART byte stream
| Outputs: Command objects (set target, set mode) or parse errors
| Dependencies: UARTProtocolConfig
| -> main.c: /* -- DD-COMP-005: CommandParser -- */
| -> main.h: /* -- DD-COMP-005: CommandParser -- */
| -> @trace DD-COMP-005 SW-REQ-003
+--

+-- DD-COMP-006 ---
| Title: UARTDiagnosticsService
| Type: Service | Pattern: Facade | Traces: SW-REQ-004
| Description: Responds to diagnostic request commands by gathering current ADC value, target
position, control output, and fault status and sending a formatted UART response.
| Inputs: Diagnostic request command, current system state
| Outputs: UART diagnostic response frame
| Dependencies: UARTDriver, SystemStateStore, FaultManager
| -> main.c: /* -- DD-COMP-006: UARTDiagnosticsService -- */
| -> main.h: /* -- DD-COMP-006: UARTDiagnosticsService -- */
| -> @trace DD-COMP-006 SW-REQ-004
+--

+-- DD-COMP-007 ---
| Title: UARTDriverConfig
| Type: Class | Pattern: Singleton | Traces: SW-REQ-005
| Description: Initializes and enforces UART settings at 115200 baud, 8 data bits, no parity, 1
stop bit.
| Inputs: Fixed UART configuration values
| Outputs: Configured UART peripheral
| Dependencies: HAL/UARTDriver
```

```
| -> main.c: /* -- DD-COMP-007: UARTDriverConfig -- */
| -> main.h: /* -- DD-COMP-007: UARTDriverConfig -- */
| -> @trace DD-COMP-007 SW-REQ-005
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 3
================================================================================
```

5. Sub-Prompt 4 - ConfigModule

Module	ConfigModule
Sub-Prompt	4 of 53
Components	DD-COMP-008, DD-COMP-009, DD-COMP-025, DD-COMP-026, DD-COMP-043, DD-COMP-049, DD-COMP-065, DD-COMP-066

```
================================================================================
SUB-PROMPT 4 OF 53
Module: ConfigModule | Project: Project-3
Components: 8
================================================================================

TASK: Implement all design components for module "ConfigModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Define data structure typedefs for all data models.
- Declare service init and run function prototypes.
- Define module-level enums, macros, and config constants.

main.c expected:
- Implement data access functions (read/write/validate).
- Implement service state machine or polling loop.
- Implement module init, process, and deinit functions.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-008 ---
```

```
| Title: ConfigService
| Type: Service | Pattern: Repository | Traces: SW-REQ-006
| Description: Provides runtime-configurable parameters for ADC sampling rate, control gains, and
UART timeouts. Supports defaults and updates via commands or stored settings.
| Inputs: Configuration requests, stored config values
| Outputs: Current configuration values
| Dependencies: ConfigStore, UARTCommandHandler
| -> main.c: /* -- DD-COMP-008: ConfigService -- */
| -> main.h: /* -- DD-COMP-008: ConfigService -- */
| -> @trace DD-COMP-008 SW-REQ-006
+--

+-- DD-COMP-009 ---
| Title: ConfigStore
| Type: Database | Pattern: Repository | Traces: SW-REQ-006
| Description: Non-volatile and/or RAM storage for configuration parameters including sampling
rate, gains, and timeouts with default values.
| Inputs: Config write values
| Outputs: Config read values
| Dependencies: NVMDriver (if non-volatile)
| -> main.c: /* -- DD-COMP-009: ConfigStore -- */
| -> main.h: /* -- DD-COMP-009: ConfigStore -- */
| -> @trace DD-COMP-009 SW-REQ-006
+--

+-- DD-COMP-025 ---
| Title: ThresholdConfigRepository
| Type: Class | Pattern: Repository | Traces: SW-REQ-017
| Description: Persists and retrieves the shipping position ADC threshold from non-volatile
memory. Provides read/write access with validation of range and checksum to ensure stored value
integrity.
| Inputs: Threshold value to persist; read request
| Outputs: Stored threshold value; status codes for read/write success or failure
| Dependencies: NvmDriver
| -> main.c: /* -- DD-COMP-025: ThresholdConfigRepository -- */
| -> main.h: /* -- DD-COMP-025: ThresholdConfigRepository -- */
| -> @trace DD-COMP-025 SW-REQ-017
+--

+-- DD-COMP-026 ---
| Title: ThresholdConfigService
| Type: Service | Pattern: Singleton | Traces: SW-REQ-017
| Description: Loads the shipping position threshold during initialization and serves it to
control logic at runtime. Falls back to default if NVM data is invalid.
```

```
| Inputs: Initialization trigger; runtime get requests
| Outputs: Active shipping threshold value
| Dependencies: ThresholdConfigRepository
| -> main.c: /* -- DD-COMP-026: ThresholdConfigService -- */
| -> main.h: /* -- DD-COMP-026: ThresholdConfigService -- */
| -> @trace DD-COMP-026 SW-REQ-017
+--

+-- DD-COMP-043 ---
| Title: Target ADC Validator
| Type: Service | Pattern: Factory | Traces: SW-REQ-032
| Description: Rejects configured target ADC values outside 500–2600 inclusive before storing or
applying them to control logic.
| Inputs: proposed_target_adc (int)
| Outputs: accepted (bool), validated_target_adc (int|null)
| Dependencies: ThresholdConfigStore
| -> main.c: /* -- DD-COMP-043: Target ADC Validator -- */
| -> main.h: /* -- DD-COMP-043: Target ADC Validator -- */
| -> @trace DD-COMP-043 SW-REQ-032
+--

+-- DD-COMP-049 ---
| Title: OvervoltageThresholdStore
| Type: Service | Pattern: Repository | Traces: SW-REQ-038
| Description: Stores and retrieves Overvoltage_Threshold from non-volatile memory and loads it
during startup for use by monitoring logic.
| Inputs: threshold_value (on write), startup_event
| Outputs: threshold_value (on read)
| Dependencies: NVMDriver
| -> main.c: /* -- DD-COMP-049: OvervoltageThresholdStore -- */
| -> main.h: /* -- DD-COMP-049: OvervoltageThresholdStore -- */
| -> @trace DD-COMP-049 SW-REQ-038
+--

+-- DD-COMP-065 ---
| Title: Voltage Configuration Store
| Type: Database | Pattern: Singleton | Traces: SW-REQ-054
| Description: Stores nominal and operating voltage configuration values used by diagnostics and
range checks. Initializes nominal voltage to 12 V.
| Inputs: nominal_voltage_value
| Outputs: NominalVoltageSetting
| Dependencies: ConfigInitializer
| -> main.c: /* -- DD-COMP-065: Voltage Configuration Store -- */
| -> main.h: /* -- DD-COMP-065: Voltage Configuration Store -- */
```

```
| -> @trace DD-COMP-065 SW-REQ-054
+--

+-- DD-COMP-066 ---
| Title: Voltage Range Configuration
| Type: Class | Pattern: N/A | Traces: SW-REQ-055
| Description: Configures acceptable supply voltage limits for operating range checks. Sets min=9
V and max=16 V for validation use.
| Inputs: min_voltage, max_voltage
| Outputs: VoltageRangeSetting
| Dependencies: VoltageConfigurationStore
| -> main.c: /* -- DD-COMP-066: Voltage Range Configuration -- */
| -> main.h: /* -- DD-COMP-066: Voltage Range Configuration -- */
| -> @trace DD-COMP-066 SW-REQ-055
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 4
================================================================================
```

6. Sub-Prompt 5 - SafetyModule

Module	SafetyModule
Sub-Prompt	5 of 53
Components	DD-COMP-010, DD-COMP-011, DD-COMP-012, DD-COMP-013, DD-COMP-014, DD-COMP-022, DD-COMP-023, DD-COMP-038, DD-COMP-040, DD-COMP-042, DD-COMP-099, DD-COMP-100, DD-COMP-101, DD-COMP-113

```
================================================================================
SUB-PROMPT 5 OF 53
Module: SafetyModule | Project: Project-3
Components: 14
================================================================================

TASK: Implement all design components for module "SafetyModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Define data structure typedefs for all data models.
- Declare algorithm function prototypes.
- Declare service init and run function prototypes.
- Define module-level enums, macros, and config constants.

main.c expected:
- Implement data access functions (read/write/validate).
- Implement algorithm logic with step-by-step inline comments.
- Implement service state machine or polling loop.
- Implement module init, process, and deinit functions.
```

```
COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-010 ---
| Title: ADCValidationService
| Type: Service | Pattern: Chain of Responsibility | Traces: SW-REQ-007
| Description: Validates ADC samples for transient faults and out-of-range values. Rejects invalid
samples, increments fault counters, and triggers safe state on persistent errors.
| Inputs: Raw ADC samples, calibrated min/max limits
| Outputs: Validated ADC samples or fault indication
| Dependencies: FaultManager, CalibrationStore
| -> main.c: /* -- DD-COMP-010: ADCValidationService -- */
| -> main.h: /* -- DD-COMP-010: ADCValidationService -- */
| -> @trace DD-COMP-010 SW-REQ-007
+--

+-- DD-COMP-011 ---
| Title: UARTFaultHandler
| Type: Service | Pattern: Strategy | Traces: SW-REQ-007
| Description: Validates UART command frames for integrity (length, checksum, value ranges) and
manages fault state for invalid or malformed data.
| Inputs: Parsed command frames, parse errors
| Outputs: Fault status updates, rejected command responses
| Dependencies: FaultManager
| -> main.c: /* -- DD-COMP-011: UARTFaultHandler -- */
| -> main.h: /* -- DD-COMP-011: UARTFaultHandler -- */
| -> @trace DD-COMP-011 SW-REQ-007
+--

+-- DD-COMP-012 ---
| Title: FaultManager
| Type: Service | Pattern: Observer | Traces: SW-REQ-007
| Description: Centralized fault state handling. Aggregates ADC and UART fault conditions and
commands entry to safe state when required.
| Inputs: Fault events, thresholds
| Outputs: Fault status, safe state command
| Dependencies: SafeStateController, SystemStateStore
| -> main.c: /* -- DD-COMP-012: FaultManager -- */
| -> main.h: /* -- DD-COMP-012: FaultManager -- */
| -> @trace DD-COMP-012 SW-REQ-007
+--

+-- DD-COMP-013 ---
| Title: SafeStateController
| Type: Service | Pattern: State | Traces: SW-REQ-008
```

```
| Description: Transitions actuator to a defined safe state (e.g., disable output or hold) when
out-of-range ADC readings are detected.
| Inputs: Safe state command, current control output
| Outputs: Actuator disable/neutral command
| Dependencies: ActuatorDriver
| -> main.c: /* -- DD-COMP-013: SafeStateController -- */
| -> main.h: /* -- DD-COMP-013: SafeStateController -- */
| -> @trace DD-COMP-013 SW-REQ-008
+--

+-- DD-COMP-014 ---
| Title: CalibrationStore
| Type: Database | Pattern: Repository | Traces: SW-REQ-008
| Description: Stores calibrated potentiometer min/max limits used to validate ADC range.
| Inputs: Calibration update values
| Outputs: Min/max ADC limits
| Dependencies: ConfigStore
| -> main.c: /* -- DD-COMP-014: CalibrationStore -- */
| -> main.h: /* -- DD-COMP-014: CalibrationStore -- */
| -> @trace DD-COMP-014 SW-REQ-008
+--

+-- DD-COMP-022 ---
| Title: Feedback Fault Monitor
| Type: Class | Pattern: State | Traces: SW-REQ-016
| Description: Monitors ADC feedback validity over time and disables actuator drive while
reporting a fault if invalid data persists beyond 50 ms.
| Inputs: Raw ADC value, validity status, time since last valid reading
| Outputs: Fault status, drive disable command
| Dependencies: ADCReadService, ControlOutputCalculator, FaultReporter
| -> main.c: /* -- DD-COMP-022: Feedback Fault Monitor -- */
| -> main.h: /* -- DD-COMP-022: Feedback Fault Monitor -- */
| -> @trace DD-COMP-022 SW-REQ-016
+--

+-- DD-COMP-023 ---
| Title: Fault Reporter
| Type: Service | Pattern: N/A | Traces: SW-REQ-016
| Description: Registers and reports system faults to diagnostics interface and logs fault
conditions for feedback invalidity.
| Inputs: Fault code, timestamp
| Outputs: Fault report entry
| Dependencies: DiagnosticsInterface, LogStorage
| -> main.c: /* -- DD-COMP-023: Fault Reporter -- */
```

```
| -> main.h: /* -- DD-COMP-023: Fault Reporter -- */
| -> @trace DD-COMP-023 SW-REQ-016
+--

+-- DD-COMP-038 ---
| Title: Actuation Range Guard
| Type: Algorithm | Pattern: Decorator | Traces: SW-REQ-027
| Description: Enforces actuation enablement only when ADC value is within 500–2600; disables or
stops actuation when outside this range.
| Inputs: latest_adc_value (int), requested_command (enum/struct)
| Outputs: guarded_command (enum/struct), actuation_allowed (bool)
| Dependencies: ADCSampleBuffer
| -> main.c: /* -- DD-COMP-038: Actuation Range Guard -- */
| -> main.h: /* -- DD-COMP-038: Actuation Range Guard -- */
| -> @trace DD-COMP-038 SW-REQ-027
+--

+-- DD-COMP-040 ---
| Title: Open-Loop Inhibit Gate
| Type: Service | Pattern: Chain of Responsibility | Traces: SW-REQ-029
| Description: Blocks actuator drive whenever valid ADC feedback is unavailable; allows drive only
if ADC is valid and fresh.
| Inputs: adc_valid (bool), requested_command (enum/struct)
| Outputs: inhibited_command (enum/struct), drive_enabled (bool)
| Dependencies: ADCValidityMonitor
| -> main.c: /* -- DD-COMP-040: Open-Loop Inhibit Gate -- */
| -> main.h: /* -- DD-COMP-040: Open-Loop Inhibit Gate -- */
| -> @trace DD-COMP-040 SW-REQ-029
+--

+-- DD-COMP-042 ---
| Title: Drive Permission Check
| Type: Algorithm | Pattern: N/A | Traces: SW-REQ-031
| Description: Permits actuator drive commands only when measured ADC value is within 500–2600
inclusive; otherwise denies drive.
| Inputs: latest_adc_value (int), requested_command (enum/struct)
| Outputs: drive_allowed (bool)
| Dependencies: ADCSampleBuffer
| -> main.c: /* -- DD-COMP-042: Drive Permission Check -- */
| -> main.h: /* -- DD-COMP-042: Drive Permission Check -- */
| -> @trace DD-COMP-042 SW-REQ-031
+--

+-- DD-COMP-099 ---
| Title: OvercurrentMonitor
```

```
| Type: Service | Pattern: Observer | Traces: SW-REQ-086
| Description: Continuously monitors motor current and triggers actuator shutdown and fault when
current exceeds configurable threshold.
| Inputs: Motor current ADC counts, overcurrent threshold, system tick
| Outputs: Actuator disable command, overcurrent fault flag
| Dependencies: AdcDriver, ConfigurationStore, ActuatorControlService, FaultManager
| -> main.c: /* -- DD-COMP-099: OvercurrentMonitor -- */
| -> main.h: /* -- DD-COMP-099: OvercurrentMonitor -- */
| -> @trace DD-COMP-099 SW-REQ-086
+--

+-- DD-COMP-100 ---
| Title: ThermalMonitor
| Type: Service | Pattern: Observer | Traces: SW-REQ-087
| Description: Monitors MCU internal temperature sensor and triggers actuator shutdown and fault
when temperature exceeds configurable threshold.
| Inputs: Temperature sensor ADC counts, thermal threshold, system tick
| Outputs: Actuator disable command, thermal fault flag
| Dependencies: AdcDriver, ConfigurationStore, ActuatorControlService, FaultManager
| -> main.c: /* -- DD-COMP-100: ThermalMonitor -- */
| -> main.h: /* -- DD-COMP-100: ThermalMonitor -- */
| -> @trace DD-COMP-100 SW-REQ-087
+--

+-- DD-COMP-101 ---
| Title: StallDetectionService
| Type: Service | Pattern: State | Traces: SW-REQ-088
| Description: Detects stall by tracking commanded motion and ensuring ADC feedback changes by at
least a configurable delta within a configurable duration; raises stall fault when not met.
| Inputs: Commanded motion state, raw ADC counts history, delta threshold, duration threshold,
system tick
| Outputs: Stall fault flag
| Dependencies: ControlStateMachine, AdcDriver, ConfigurationStore, FaultManager, SystemTimer
| -> main.c: /* -- DD-COMP-101: StallDetectionService -- */
| -> main.h: /* -- DD-COMP-101: StallDetectionService -- */
| -> @trace DD-COMP-101 SW-REQ-088
+--

+-- DD-COMP-113 ---
| Title: ShippingLimitGuard
| Type: Class | Pattern: Decorator | Traces: SW-REQ-098
| Description: Checks ADC feedback against the configured Shipping_ADC_Value threshold and
prevents actuator motion toward the limit by issuing a stop command and inhibiting further
commands in the limit direction.
```

```
| Inputs: ADC value, Shipping_ADC_Value threshold, requested actuator direction
| Outputs: Stop/inhibit flags, modified actuator command
| Dependencies: ConfigStore, ActuatorDriver
| -> main.c: /* -- DD-COMP-113: ShippingLimitGuard -- */
| -> main.h: /* -- DD-COMP-113: ShippingLimitGuard -- */
| -> @trace DD-COMP-113 SW-REQ-098
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 5
================================================================================
```

7. Sub-Prompt 6 - ComplianceModule

Module	ComplianceModule
Sub-Prompt	6 of 53
Components	DD-COMP-015

```
================================================================================
SUB-PROMPT 6 OF 53
Module: ComplianceModule | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "ComplianceModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Define module-level enums, macros, and config constants.

main.c expected:
- Implement module init, process, and deinit functions.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-015 ---
| Title: MISRA Compliance Process
| Type: Module | Pattern: N/A | Traces: SW-REQ-009
| Description: Defines project-wide MISRA-C:2012 compliance workflow including static analysis
configuration, rule enforcement, and deviation recording. Integrates tooling outputs into build
artifacts and maintains a deviation register for approved exceptions.
```

```
| Inputs: Source code files, MISRA rule set, approved deviation list
| Outputs: Static analysis reports, deviation documentation artifacts
| Dependencies: BuildSystem, StaticAnalysisToolchain, ConfigurationRepository
| -> main.c: /* -- DD-COMP-015: MISRA Compliance Process -- */
| -> main.h: /* -- DD-COMP-015: MISRA Compliance Process -- */
| -> @trace DD-COMP-015 SW-REQ-009
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 6
================================================================================
```

8. Sub-Prompt 7 - HALModule

Module	HALModule
Sub-Prompt	7 of 53
Components	DD-COMP-017, DD-COMP-018, DD-COMP-024

```
================================================================================
SUB-PROMPT 7 OF 53
Module: HALModule | Project: Project-3
Components: 3
================================================================================

TASK: Implement all design components for module "HALModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.

main.c expected:
- Implement service state machine or polling loop.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-017 ---
| Title: ADC Channel Configuration
| Type: Service | Pattern: Factory | Traces: SW-REQ-011
| Description: Configures a single ADC channel for actuator potentiometer feedback with required
sampling time, resolution, and input pin mapping.
| Inputs: ADC channel ID, pin configuration, sampling settings
```

```
| Outputs: Initialized ADC channel handle
| Dependencies: MCU_ADC_Driver, ConfigurationRepository
| -> main.c: /* -- DD-COMP-017: ADC Channel Configuration -- */
| -> main.h: /* -- DD-COMP-017: ADC Channel Configuration -- */
| -> @trace DD-COMP-017 SW-REQ-011
+--

+-- DD-COMP-018 ---
| Title: ADC Read Service
| Type: Service | Pattern: N/A | Traces: SW-REQ-012
| Description: Reads the configured ADC channel for actuator feedback and provides raw ADC counts.
Invoked by control loop each cycle.
| Inputs: ADC channel handle
| Outputs: Raw ADC value (integer counts)
| Dependencies: MCU_ADC_Driver
| -> main.c: /* -- DD-COMP-018: ADC Read Service -- */
| -> main.h: /* -- DD-COMP-018: ADC Read Service -- */
| -> @trace DD-COMP-018 SW-REQ-012
+--

+-- DD-COMP-024 ---
| Title: Actuator Drive Output
| Type: Service | Pattern: N/A | Traces: SW-REQ-014
| Description: Applies the computed control output to the actuator driver hardware and supports
disable command from safety module.
| Inputs: Control output command, disable flag
| Outputs: Updated actuator drive signal
| Dependencies: MCU_PWM_Driver, ControlOutputCalculator, FeedbackFaultMonitor
| -> main.c: /* -- DD-COMP-024: Actuator Drive Output -- */
| -> main.h: /* -- DD-COMP-024: Actuator Drive Output -- */
| -> @trace DD-COMP-024 SW-REQ-014
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
==============================================================================
END SUB-PROMPT 7
==============================================================================
```

9. Sub-Prompt 8 - SignalProcessingModule

Module	SignalProcessingModule
Sub-Prompt	8 of 53
Components	DD-COMP-019

```
================================================================================
SUB-PROMPT 8 OF 53
Module: SignalProcessingModule | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "SignalProcessingModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare algorithm function prototypes.

main.c expected:
- Implement algorithm logic with step-by-step inline comments.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-019 ---
| Title: Position Scaling Algorithm
| Type: Algorithm | Pattern: N/A | Traces: SW-REQ-013
| Description: Scales raw ADC counts to normalized position in [0.0, 1.0] using configurable
min/max calibration values and clamps values outside range.
| Inputs: Raw ADC value, calibration_min, calibration_max
```

```
| Outputs: Normalized position (float 0.0 to 1.0)
| Dependencies: CalibrationRepository
| -> main.c: /* -- DD-COMP-019: Position Scaling Algorithm -- */
| -> main.h: /* -- DD-COMP-019: Position Scaling Algorithm -- */
| -> @trace DD-COMP-019 SW-REQ-013
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 8
================================================================================
```

10. Sub-Prompt 9 - SchedulerModule

Module	SchedulerModule
Sub-Prompt	9 of 53
Components	DD-COMP-021

```
================================================================================
SUB-PROMPT 9 OF 53
Module: SchedulerModule | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "SchedulerModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.

main.c expected:
- Implement service state machine or polling loop.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-021 ---
| Title: Control Loop Scheduler
| Type: Service | Pattern: Observer | Traces: SW-REQ-015
| Description: Executes the control loop at a fixed period of 10 ms with allowable jitter of ±1
ms, triggering ADC read, scaling, control calculation, and output update.
| Inputs: System tick timer interrupts
```

```
| Outputs: Periodic control loop execution events
| Dependencies: MCU_Timer_Driver, ControlOutputCalculator, ADCReadService
| -> main.c: /* -- DD-COMP-021: Control Loop Scheduler -- */
| -> main.h: /* -- DD-COMP-021: Control Loop Scheduler -- */
| -> @trace DD-COMP-021 SW-REQ-015
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 9
================================================================================
```

11. Sub-Prompt 10 - UartModule

Module	UartModule
Sub-Prompt	10 of 53
Components	DD-COMP-028, DD-COMP-029, DD-COMP-032, DD-COMP-058, DD-COMP-059

```
================================================================================
SUB-PROMPT 10 OF 53
Module: UartModule | Project: Project-3
Components: 5
================================================================================

TASK: Implement all design components for module "UartModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare algorithm function prototypes.
- Declare service init and run function prototypes.

main.c expected:
- Implement algorithm logic with step-by-step inline comments.
- Implement service state machine or polling loop.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-028 ---
| Title: UartCommandParser
| Type: Service | Pattern: Observer | Traces: SW-REQ-019
```

```
| Description: Parses incoming UART frames, validates command format, and emits ON/OFF command
events. Ensures receipt-to-dispatch latency within 50 ms.
| Inputs: UART RX bytes/frames
| Outputs: Command event (ON/OFF) with timestamp; validation result
| Dependencies: UartDriver, ActuatorStateMachine
| -> main.c: /* -- DD-COMP-028: UartCommandParser -- */
| -> main.h: /* -- DD-COMP-028: UartCommandParser -- */
| -> @trace DD-COMP-028 SW-REQ-019
+--

+-- DD-COMP-029 ---
| Title: UartCommandParser_OFF
| Type: Service | Pattern: Observer | Traces: SW-REQ-020
| Description: Handles OFF command parsing and dispatch in the same parsing service to trigger OFF
transitions within 50 ms.
| Inputs: UART RX bytes/frames
| Outputs: OFF command event with timestamp
| Dependencies: UartDriver, ActuatorStateMachine
| -> main.c: /* -- DD-COMP-029: UartCommandParser_OFF -- */
| -> main.h: /* -- DD-COMP-029: UartCommandParser_OFF -- */
| -> @trace DD-COMP-029 SW-REQ-020
+--

+-- DD-COMP-032 ---
| Title: UartAckService
| Type: Service | Pattern: Observer | Traces: SW-REQ-021
| Description: Sends UART acknowledgement frames for valid ON/OFF commands within 20 ms of command
processing. Subscribes to state change or command validation events.
| Inputs: Validated command event or state change event
| Outputs: UART ACK message
| Dependencies: UartDriver, UartCommandParser, ActuatorStateMachine
| -> main.c: /* -- DD-COMP-032: UartAckService -- */
| -> main.h: /* -- DD-COMP-032: UartAckService -- */
| -> @trace DD-COMP-032 SW-REQ-021
+--

+-- DD-COMP-058 ---
| Title: UartActuatorCommandInterface
| Type: Service | Pattern: Facade | Traces: SW-REQ-047
| Description: Receives and parses actuator control commands over UART when link is active and
forwards valid commands to the command gate.
| Inputs: UART byte stream; link active status
| Outputs: Parsed actuator command objects; error responses
| Dependencies: UartDriver; CommandParser; ActuatorCommandGate; LinkMonitor
```

```
| -> main.c: /* -- DD-COMP-058: UartActuatorCommandInterface -- */
| -> main.h: /* -- DD-COMP-058: UartActuatorCommandInterface -- */
| -> @trace DD-COMP-058 SW-REQ-047
+--

+-- DD-COMP-059 ---
| Title: ActuatorOnCommandParser
| Type: Algorithm | Pattern: Factory | Traces: SW-REQ-048
| Description: Recognizes the UART command token "ACTUATOR_ON" and maps it to an actuator enable
command object with required parameters.
| Inputs: Parsed UART command string
| Outputs: ActuatorCommand object of type ACTUATOR_ON or invalid command result
| Dependencies: CommandParser; CommandRegistry
| -> main.c: /* -- DD-COMP-059: ActuatorOnCommandParser -- */
| -> main.h: /* -- DD-COMP-059: ActuatorOnCommandParser -- */
| -> @trace DD-COMP-059 SW-REQ-048
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 10
================================================================================
```

12. Sub-Prompt 11 - AdcModule

Module	AdcModule
Sub-Prompt	11 of 53
Components	DD-COMP-033, DD-COMP-034, DD-COMP-035

```
================================================================================
SUB-PROMPT 11 OF 53
Module: AdcModule | Project: Project-3
Components: 3
================================================================================

TASK: Implement all design components for module "AdcModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.
- Define module-level enums, macros, and config constants.

main.c expected:
- Implement service state machine or polling loop.
- Implement module init, process, and deinit functions.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-033 ---
| Title: AdcDriverConfig
| Type: Class | Pattern: Facade | Traces: SW-REQ-023
```

```
| Description: Configures the internal ADC hardware to 12-bit resolution or higher and applies
sampling rate settings. Provides initialization API called at startup.
| Inputs: Initialization trigger; configuration parameters
| Outputs: Configured ADC peripheral; status code
| Dependencies: HardwareAbstractionLayer
| -> main.c: /* -- DD-COMP-033: AdcDriverConfig -- */
| -> main.h: /* -- DD-COMP-033: AdcDriverConfig -- */
| -> @trace DD-COMP-033 SW-REQ-023
+--

+-- DD-COMP-034 ---
| Title: AdcSamplingScheduler
| Type: Service | Pattern: Producer-Consumer | Traces: SW-REQ-024
| Description: Schedules continuous ADC sampling at >=1000 samples per second using timer
interrupt or DMA. Ensures no gaps longer than 2 ms by using a periodic trigger and buffering.
| Inputs: Timer ticks; ADC conversion complete interrupts
| Outputs: ADC sample stream into ring buffer
| Dependencies: AdcDriverConfig, TimerDriver, RingBuffer
| -> main.c: /* -- DD-COMP-034: AdcSamplingScheduler -- */
| -> main.h: /* -- DD-COMP-034: AdcSamplingScheduler -- */
| -> @trace DD-COMP-034 SW-REQ-024
+--

+-- DD-COMP-035 ---
| Title: AdcContinuousAcquisition
| Type: Service | Pattern: Observer | Traces: SW-REQ-022
| Description: Continuously acquires ADC samples during normal operation and feeds them to control
logic. Monitors timing to ensure no gaps >2 ms and raises diagnostic flags if violated.
| Inputs: ADC conversion complete events; system state
| Outputs: Sample notifications to control logic; diagnostic status
| Dependencies: AdcSamplingScheduler, ShippingPositionStopLogic
| -> main.c: /* -- DD-COMP-035: AdcContinuousAcquisition -- */
| -> main.h: /* -- DD-COMP-035: AdcContinuousAcquisition -- */
| -> @trace DD-COMP-035 SW-REQ-022
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
```

==========

END SUB-PROMPT 11

==========

13. Sub-Prompt 12 - SensingModule

Module	SensingModule
Sub-Prompt	12 of 53
Components	DD-COMP-036, DD-COMP-039, DD-COMP-041

```
================================================================================
SUB-PROMPT 12 OF 53
Module: SensingModule | Project: Project-3
Components: 3
================================================================================

TASK: Implement all design components for module "SensingModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare algorithm function prototypes.
- Declare service init and run function prototypes.
- Define module-level enums, macros, and config constants.

main.c expected:
- Implement algorithm logic with step-by-step inline comments.
- Implement service state machine or polling loop.
- Implement module init, process, and deinit functions.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-036 ---
| Title: ADC Sample Buffer
```

```
| Type: Class | Pattern: Singleton | Traces: SW-REQ-025
| Description: Maintains the latest ADC sample and timestamp; provides atomic read access to the
most recent sample for control computations.
| Inputs: raw_adc_value (int), sample_timestamp_us (uint64)
| Outputs: latest_adc_value (int), latest_timestamp_us (uint64)
| Dependencies: ADCDriver
| -> main.c: /* -- DD-COMP-036: ADC Sample Buffer -- */
| -> main.h: /* -- DD-COMP-036: ADC Sample Buffer -- */
| -> @trace DD-COMP-036 SW-REQ-025
+--

+-- DD-COMP-039 ---
| Title: Real-Time ADC Freshness Check
| Type: Algorithm | Pattern: N/A | Traces: SW-REQ-028
| Description: Validates that control decisions use a sample not older than 2 ms by comparing
current time with latest sample timestamp before control execution.
| Inputs: current_time_us (uint64), latest_timestamp_us (uint64)
| Outputs: is_fresh (bool)
| Dependencies: ADCSampleBuffer, SystemClock
| -> main.c: /* -- DD-COMP-039: Real-Time ADC Freshness Check -- */
| -> main.h: /* -- DD-COMP-039: Real-Time ADC Freshness Check -- */
| -> @trace DD-COMP-039 SW-REQ-028
+--

+-- DD-COMP-041 ---
| Title: ADC Validity Monitor
| Type: Service | Pattern: Observer | Traces: SW-REQ-030
| Description: Flags ADC feedback invalid if no new sample arrives within 2 ms or if value is
outside 0–4095; updates validity state.
| Inputs: latest_adc_value (int), latest_timestamp_us (uint64), current_time_us (uint64)
| Outputs: adc_valid (bool), validity_reason (enum)
| Dependencies: ADCSampleBuffer, SystemClock
| -> main.c: /* -- DD-COMP-041: ADC Validity Monitor -- */
| -> main.h: /* -- DD-COMP-041: ADC Validity Monitor -- */
| -> @trace DD-COMP-041 SW-REQ-030
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)
```

```
Pure C code only. No explanations. No markdown.
==============================================================================
END SUB-PROMPT 12
==============================================================================
```

14. Sub-Prompt 13 - ControlLoopModule

Module	ControlLoopModule
Sub-Prompt	13 of 53
Components	DD-COMP-044, DD-COMP-045, DD-COMP-051

```
================================================================================
SUB-PROMPT 13 OF 53
Module: ControlLoopModule | Project: Project-3
Components: 3
================================================================================

TASK: Implement all design components for module "ControlLoopModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Define module-level enums, macros, and config constants.

main.c expected:
- Implement module init, process, and deinit functions.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-044 ---
| Title: ActuatorSafetyMonitor
| Type: Class | Pattern: Observer | Traces: SW-REQ-033
| Description: Evaluates ADC bounds each control loop cycle and disables actuator drive when the
measured ADC value is <500 or >2600. Executes within the loop timing budget to ensure action
within one cycle.
```

```
| Inputs: measured_adc_value, control_loop_tick
| Outputs: drive_enable=false when out of range
| Dependencies: ActuatorDriveService, ADCService
| -> main.c: /* -- DD-COMP-044: ActuatorSafetyMonitor -- */
| -> main.h: /* -- DD-COMP-044: ActuatorSafetyMonitor -- */
| -> @trace DD-COMP-044 SW-REQ-033
+--

+-- DD-COMP-045 ---
| Title: ActuatorTargetStopController
| Type: Class | Pattern: Strategy | Traces: SW-REQ-034
| Description: Compares measured ADC to configured target and stops actuator movement when within
±1 count. Issues stop command and inhibits further drive until outside tolerance or new command.
| Inputs: measured_adc_value, target_adc_value
| Outputs: drive_command=STOP
| Dependencies: ActuatorDriveService, TargetConfigService
| -> main.c: /* -- DD-COMP-045: ActuatorTargetStopController -- */
| -> main.h: /* -- DD-COMP-045: ActuatorTargetStopController -- */
| -> @trace DD-COMP-045 SW-REQ-034
+--

+-- DD-COMP-051 ---
| Title: OvervoltageDriveDisabler
| Type: Class | Pattern: Observer | Traces: SW-REQ-040
| Description: Disables actuator drive within one control loop cycle of overvoltage detection by
acting on the overvoltage flag in the loop.
| Inputs: overvoltage_flag, control_loop_tick
| Outputs: drive_enable=false
| Dependencies: OvervoltageDetector, ActuatorDriveService
| -> main.c: /* -- DD-COMP-051: OvervoltageDriveDisabler -- */
| -> main.h: /* -- DD-COMP-051: OvervoltageDriveDisabler -- */
| -> @trace DD-COMP-051 SW-REQ-040
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
==============================================================================
END SUB-PROMPT 13
```

==========

15. Sub-Prompt 14 - HardwareAbstractionLayer

Module	HardwareAbstractionLayer
Sub-Prompt	14 of 53
Components	DD-COMP-046

```
================================================================================
SUB-PROMPT 14 OF 53
Module: HardwareAbstractionLayer | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "HardwareAbstractionLayer".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.

main.c expected:
- Implement service state machine or polling loop.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-046 ---
| Title: SupplyVoltageADCChannelConfig
| Type: Service | Pattern: Facade | Traces: SW-REQ-035
| Description: Initializes and configures the dedicated ADC channel connected to the external
supply voltage divider with proper sampling time and reference settings.
| Inputs: adc_channel_id, sampling_time, reference_voltage
```

```
| Outputs: configured_adc_channel_handle
| Dependencies: ADCDriver
| -> main.c: /* -- DD-COMP-046: SupplyVoltageADCChannelConfig -- */
| -> main.h: /* -- DD-COMP-046: SupplyVoltageADCChannelConfig -- */
| -> @trace DD-COMP-046 SW-REQ-035
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 14
================================================================================
```

16. Sub-Prompt 15 - VoltageMonitorModule

Module	VoltageMonitorModule
Sub-Prompt	15 of 53
Components	DD-COMP-047, DD-COMP-048, DD-COMP-050

```
================================================================================
SUB-PROMPT 15 OF 53
Module: VoltageMonitorModule | Project: Project-3
Components: 3
================================================================================

TASK: Implement all design components for module "VoltageMonitorModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare algorithm function prototypes.
- Declare service init and run function prototypes.
- Define module-level enums, macros, and config constants.

main.c expected:
- Implement algorithm logic with step-by-step inline comments.
- Implement service state machine or polling loop.
- Implement module init, process, and deinit functions.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-047 ---
| Title: SupplyVoltageScaler
```

```
| Type: Algorithm | Pattern: N/A | Traces: SW-REQ-036
| Description: Computes external supply voltage by applying the configured divider ratio to the
raw ADC value and ADC reference scaling.
| Inputs: raw_adc_value, divider_ratio, adc_ref_voltage, adc_resolution
| Outputs: computed_supply_voltage
| Dependencies: SupplyVoltageADCChannelConfig
| -> main.c: /* -- DD-COMP-047: SupplyVoltageScaler -- */
| -> main.h: /* -- DD-COMP-047: SupplyVoltageScaler -- */
| -> @trace DD-COMP-047 SW-REQ-036
+--

+-- DD-COMP-048 ---
| Title: SupplyVoltageSampler
| Type: Service | Pattern: Observer | Traces: SW-REQ-037
| Description: Samples the supply ADC channel at least once per control loop cycle and updates the
computed supply voltage cache.
| Inputs: control_loop_tick
| Outputs: updated_supply_voltage_cache
| Dependencies: ADCService, SupplyVoltageScaler
| -> main.c: /* -- DD-COMP-048: SupplyVoltageSampler -- */
| -> main.h: /* -- DD-COMP-048: SupplyVoltageSampler -- */
| -> @trace DD-COMP-048 SW-REQ-037
+--

+-- DD-COMP-050 ---
| Title: OvervoltageDetector
| Type: Class | Pattern: Observer | Traces: SW-REQ-039
| Description: Compares computed supply voltage against Overvoltage_Threshold and raises an
overvoltage condition when voltage >= threshold.
| Inputs: computed_supply_voltage, overvoltage_threshold
| Outputs: overvoltage_flag
| Dependencies: SupplyVoltageSampler, OvervoltageThresholdStore
| -> main.c: /* -- DD-COMP-050: OvervoltageDetector -- */
| -> main.h: /* -- DD-COMP-050: OvervoltageDetector -- */
| -> @trace DD-COMP-050 SW-REQ-039
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)
```

```
Pure C code only. No explanations. No markdown.
==============================================================================
END SUB-PROMPT 15
==============================================================================
```

17. Sub-Prompt 16 - FaultManagementModule

Module	FaultManagementModule
Sub-Prompt	16 of 53
Components	DD-COMP-053, DD-COMP-055, DD-COMP-056

```
================================================================================
SUB-PROMPT 16 OF 53
Module: FaultManagementModule | Project: Project-3
Components: 3
================================================================================

TASK: Implement all design components for module "FaultManagementModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.
- Define module-level enums, macros, and config constants.

main.c expected:
- Implement service state machine or polling loop.
- Implement module init, process, and deinit functions.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-053 ---
| Title: FaultTransitionHandler
| Type: Class | Pattern: Observer | Traces: SW-REQ-042
```

```
| Description: Listens for overvoltage detection events and triggers an immediate transition to
FAULT state within the next control loop cycle.
| Inputs: Overvoltage event; current system state; control loop tick
| Outputs: State transition request to FAULT
| Dependencies: OvervoltageMonitor; StateMachine; ControlLoopScheduler
| -> main.c: /* -- DD-COMP-053: FaultTransitionHandler -- */
| -> main.h: /* -- DD-COMP-053: FaultTransitionHandler -- */
| -> @trace DD-COMP-053 SW-REQ-042
+--

+-- DD-COMP-055 ---
| Title: FaultStateRetentionLogic
| Type: Class | Pattern: State | Traces: SW-REQ-044
| Description: Enforces FAULT state retention until voltage is within limits and a reset or
authorized UART clear command is received. Validates both conditions before allowing exit.
| Inputs: Current voltage status; reset signal; UART clear fault command; current system state
| Outputs: State transition request to exit FAULT or retain FAULT
| Dependencies: OvervoltageMonitor; StateMachine; UartCommandHandler; ResetManager
| -> main.c: /* -- DD-COMP-055: FaultStateRetentionLogic -- */
| -> main.h: /* -- DD-COMP-055: FaultStateRetentionLogic -- */
| -> @trace DD-COMP-055 SW-REQ-044
+--

+-- DD-COMP-056 ---
| Title: FaultExitVoltageChecker
| Type: Service | Pattern: Scheduler | Traces: SW-REQ-045
| Description: Periodically evaluates supply voltage against acceptable limits at least every 100
ms while in FAULT state and updates eligibility to exit FAULT.
| Inputs: Periodic timer tick; voltage sample; acceptable voltage limits
| Outputs: Voltage-within-limits flag
| Dependencies: VoltageSampler; ConfigurationService; TimerService
| -> main.c: /* -- DD-COMP-056: FaultExitVoltageChecker -- */
| -> main.h: /* -- DD-COMP-056: FaultExitVoltageChecker -- */
| -> @trace DD-COMP-056 SW-REQ-045
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
```

==========

END SUB-PROMPT 16

==========

18. Sub-Prompt 17 - DiagnosticsModule

Module	DiagnosticsModule
Sub-Prompt	17 of 53
Components	DD-COMP-054

```
================================================================================
SUB-PROMPT 17 OF 53
Module: DiagnosticsModule | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "DiagnosticsModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.

main.c expected:
- Implement service state machine or polling loop.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-054 ---
| Title: OvervoltageUartNotifier
| Type: Service | Pattern: Observer | Traces: SW-REQ-043
| Description: Sends UART diagnostic message "OVERVOLTAGE_FAULT_ACTIVE" within 100 ms of detecting
voltage >= Overvoltage_Threshold. Uses a debounce to avoid repeated transmissions until condition
clears.
```

```
| Inputs: Overvoltage status change; configured Overvoltage_Threshold
| Outputs: UART message string
| Dependencies: OvervoltageMonitor; UartDriver; ConfigurationService; TimerService
| -> main.c: /* -- DD-COMP-054: OvervoltageUartNotifier -- */
| -> main.h: /* -- DD-COMP-054: OvervoltageUartNotifier -- */
| -> @trace DD-COMP-054 SW-REQ-043
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 17
================================================================================
```

19. Sub-Prompt 18 - PowerSafetyModule

Module	PowerSafetyModule
Sub-Prompt	18 of 53
Components	DD-COMP-057

```
================================================================================
SUB-PROMPT 18 OF 53
Module: PowerSafetyModule | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "PowerSafetyModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.

main.c expected:
- Implement service state machine or polling loop.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-057 ---
| Title: OvervoltageMonitor
| Type: Service | Pattern: Observer | Traces: SW-REQ-046
| Description: Samples supply voltage and evaluates overvoltage status at least every 100 ms
across all active states. Publishes status changes to subscribers.
| Inputs: Voltage samples; Overvoltage_Threshold; acceptable limits; periodic timer tick
```

```
| Outputs: Overvoltage status flag; voltage-within-limits flag; event notifications
| Dependencies: VoltageSampler; ConfigurationService; TimerService; EventBus
| -> main.c: /* -- DD-COMP-057: OvervoltageMonitor -- */
| -> main.h: /* -- DD-COMP-057: OvervoltageMonitor -- */
| -> @trace DD-COMP-057 SW-REQ-046
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 18
================================================================================
```

20. Sub-Prompt 19 - UartCommModule

Module	UartCommModule
Sub-Prompt	19 of 53
Components	DD-COMP-060

```
================================================================================
SUB-PROMPT 19 OF 53
Module: UartCommModule | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "UartCommModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Define module-level enums, macros, and config constants.

main.c expected:
- Implement module init, process, and deinit functions.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-060 ---
| Title: UART Command Parser
| Type: Class | Pattern: N/A | Traces: SW-REQ-049
| Description: Parses raw UART input into command tokens and raw string payloads. Recognizes
supported command strings including ACTUATOR_OFF and ACTUATOR_ON and forwards parsed results to
the command dispatcher.
```

```
| Inputs: raw_uart_bytes, message_terminator
| Outputs: ParsedCommand {command_name, raw_string}
| Dependencies: UartDriver, CommandDispatcher
| -> main.c: /* -- DD-COMP-060: UART Command Parser -- */
| -> main.h: /* -- DD-COMP-060: UART Command Parser -- */
| -> @trace DD-COMP-060 SW-REQ-049
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 19
================================================================================
```

21. Sub-Prompt 20 - CommandModule

Module	CommandModule
Sub-Prompt	20 of 53
Components	DD-COMP-061, DD-COMP-063

```
================================================================================
SUB-PROMPT 20 OF 53
Module: CommandModule | Project: Project-3
Components: 2
================================================================================

TASK: Implement all design components for module "CommandModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.

main.c expected:
- Implement service state machine or polling loop.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-061 ---
| Title: Command Dispatcher
| Type: Service | Pattern: Command | Traces: SW-REQ-050
| Description: Routes parsed commands to appropriate handlers. If command is ACTUATOR_ON, triggers
actuator control logic within timing constraints; if ACTUATOR_OFF, triggers actuator disable;
otherwise routes to invalid command handler without changing actuator state.
```

```
| Inputs: ParsedCommand
| Outputs: DispatchResult {status, handler_invoked}
| Dependencies: ActuatorController, InvalidCommandHandler
| -> main.c: /* -- DD-COMP-061: Command Dispatcher -- */
| -> main.h: /* -- DD-COMP-061: Command Dispatcher -- */
| -> @trace DD-COMP-061 SW-REQ-050
+--

+-- DD-COMP-063 ---
| Title: Invalid Command Handler
| Type: Service | Pattern: N/A | Traces: SW-REQ-052
| Description: Handles invalid UART commands by ensuring no actuator state change occurs and by
invoking the logging service with timestamp and raw command string.
| Inputs: ParsedCommand.raw_string
| Outputs: HandledResult {ignored:true}
| Dependencies: ActuatorController, LoggingService, TimeService
| -> main.c: /* -- DD-COMP-063: Invalid Command Handler -- */
| -> main.h: /* -- DD-COMP-063: Invalid Command Handler -- */
| -> @trace DD-COMP-063 SW-REQ-052
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 20
================================================================================
```

22. Sub-Prompt 21 - ActuatorModule

Module	ActuatorModule
Sub-Prompt	21 of 53
Components	DD-COMP-062

```
================================================================================
SUB-PROMPT 21 OF 53
Module: ActuatorModule | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "ActuatorModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.

main.c expected:
- Implement service state machine or polling loop.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-062 ---
| Title: Actuator Controller
| Type: Service | Pattern: Facade | Traces: SW-REQ-051
| Description: Implements actuator enable and disable operations. Executes configured control
logic for ACTUATOR_ON and disables GPIO/PWM outputs for ACTUATOR_OFF within specified time limits.
| Inputs: command_name, control_config
```

```
| Outputs: ActuatorState {enabled, timestamp}
| Dependencies: GpioDriver, PwmDriver, ControlLogic
| -> main.c: /* -- DD-COMP-062: Actuator Controller -- */
| -> main.h: /* -- DD-COMP-062: Actuator Controller -- */
| -> @trace DD-COMP-062 SW-REQ-051
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 21
================================================================================
```

23. Sub-Prompt 22 - LoggingModule

Module	LoggingModule
Sub-Prompt	22 of 53
Components	DD-COMP-064

```
================================================================================
SUB-PROMPT 22 OF 53
Module: LoggingModule | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "LoggingModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.

main.c expected:
- Implement service state machine or polling loop.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-064 ---
| Title: Invalid Command Logger
| Type: Service | Pattern: Repository | Traces: SW-REQ-053
| Description: Logs invalid UART commands with timestamp and raw command string to persistent or
ring-buffer storage.
| Inputs: timestamp, raw_command_string
```

```
| Outputs: LogEntryId
| Dependencies: TimeService, LogStorage
| -> main.c: /* -- DD-COMP-064: Invalid Command Logger -- */
| -> main.h: /* -- DD-COMP-064: Invalid Command Logger -- */
| -> @trace DD-COMP-064 SW-REQ-053
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 22
================================================================================
```

24. Sub-Prompt 23 - StartupModule

Module	StartupModule
Sub-Prompt	23 of 53
Components	DD-COMP-067, DD-COMP-068

```
================================================================================
SUB-PROMPT 23 OF 53
Module: StartupModule | Project: Project-3
Components: 2
================================================================================

TASK: Implement all design components for module "StartupModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.
- Define module-level enums, macros, and config constants.

main.c expected:
- Implement service state machine or polling loop.
- Implement module init, process, and deinit functions.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-067 ---
| Title: Startup Safety Manager
| Type: Service | Pattern: State | Traces: SW-REQ-056
```

```
| Description: Ensures all actuator control outputs (PA5 GPIO and PWM outputs) are disabled or
high-impedance until initialization completes and a 100 ms post-reset stabilization delay has
elapsed. Enables outputs only after delay and initialization success.
| Inputs: reset_event, init_complete_event
| Outputs: SafetyState {outputs_enabled}
| Dependencies: GpioDriver, PwmDriver, TimeService, SystemInitializer
| -> main.c: /* -- DD-COMP-067: Startup Safety Manager -- */
| -> main.h: /* -- DD-COMP-067: Startup Safety Manager -- */
| -> @trace DD-COMP-067 SW-REQ-056
+--

+-- DD-COMP-068 ---
| Title: StartupDelayManager
| Type: Class | Pattern: State | Traces: SW-REQ-057
| Description: Implements a post-reset delay gate. On system reset, it starts a hardware timer and
blocks control loop scheduling and actuator enable until the elapsed time is at least 50 ms, then
signals readiness.
| Inputs: Reset event, system tick/timer count
| Outputs: ready_to_enable boolean, enable_control_loop event
| Dependencies: SystemTimerService, ControlLoopScheduler, ActuatorControlService
| -> main.c: /* -- DD-COMP-068: StartupDelayManager -- */
| -> main.h: /* -- DD-COMP-068: StartupDelayManager -- */
| -> @trace DD-COMP-068 SW-REQ-057
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 23
================================================================================
```

25. Sub-Prompt 24 - ADCModule

Module	ADCModule
Sub-Prompt	24 of 53
Components	DD-COMP-069, DD-COMP-070, DD-COMP-071, DD-COMP-072

```
================================================================================
SUB-PROMPT 24 OF 53
Module: ADCModule | Project: Project-3
Components: 4
================================================================================

TASK: Implement all design components for module "ADCModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare algorithm function prototypes.
- Declare service init and run function prototypes.

main.c expected:
- Implement algorithm logic with step-by-step inline comments.
- Implement service state machine or polling loop.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-069 ---
| Title: ADCResolutionConfigurator
| Type: Service | Pattern: N/A | Traces: SW-REQ-058
```

```
| Description: Configures the ADC peripheral registers to 12-bit resolution during initialization,
verifying the setting and reporting status.
| Inputs: Init call, desired resolution constant (12-bit)
| Outputs: Status code (OK/ERROR)
| Dependencies: MCU_HAL_ADC
| -> main.c: /* -- DD-COMP-069: ADCResolutionConfigurator -- */
| -> main.h: /* -- DD-COMP-069: ADCResolutionConfigurator -- */
| -> @trace DD-COMP-069 SW-REQ-058
+--

+-- DD-COMP-070 ---
| Title: ADCReferenceConfigurator
| Type: Service | Pattern: N/A | Traces: SW-REQ-059
| Description: Sets ADC reference voltage to 3.3 V by configuring the reference source selection
and validates it against supported reference options.
| Inputs: Init call, reference selection (3.3V internal/external)
| Outputs: Status code (OK/ERROR)
| Dependencies: MCU_HAL_ADC, BoardConfig
| -> main.c: /* -- DD-COMP-070: ADCReferenceConfigurator -- */
| -> main.h: /* -- DD-COMP-070: ADCReferenceConfigurator -- */
| -> @trace DD-COMP-070 SW-REQ-059
+--

+-- DD-COMP-071 ---
| Title: ADCSamplingScheduler
| Type: Service | Pattern: Observer | Traces: SW-REQ-060
| Description: Schedules ADC sampling at a minimum of 1 kHz using a hardware timer trigger or
periodic task. Ensures sample period <= 1 ms and dispatches samples to processing pipeline.
| Inputs: System clock, desired sample rate
| Outputs: ADC sample ready events, sample timestamps
| Dependencies: SystemTimerService, MCU_HAL_ADC, ADCDriver
| -> main.c: /* -- DD-COMP-071: ADCSamplingScheduler -- */
| -> main.h: /* -- DD-COMP-071: ADCSamplingScheduler -- */
| -> @trace DD-COMP-071 SW-REQ-060
+--

+-- DD-COMP-072 ---
| Title: ADCSaturationDetector
| Type: Algorithm | Pattern: State | Traces: SW-REQ-061
| Description: Tracks consecutive ADC samples equal to maximum count. If 5 consecutive maximum
readings are observed, flags saturation and requests transition to FAULT state.
| Inputs: ADC sample value, ADC max count
| Outputs: saturation_detected boolean, fault_request event
| Dependencies: StateManager, ADCDriver
```

```
| -> main.c: /* -- DD-COMP-072: ADCSaturationDetector -- */
| -> main.h: /* -- DD-COMP-072: ADCSaturationDetector -- */
| -> @trace DD-COMP-072 SW-REQ-061
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 24
================================================================================
```

26. Sub-Prompt 25 - BoardSupportModule

Module	BoardSupportModule
Sub-Prompt	25 of 53
Components	DD-COMP-073, DD-COMP-074

```
================================================================================
SUB-PROMPT 25 OF 53
Module: BoardSupportModule | Project: Project-3
Components: 2
================================================================================

TASK: Implement all design components for module "BoardSupportModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.

main.c expected:
- Implement service state machine or polling loop.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-073 ---
| Title: ADCPinInitializer
| Type: Service | Pattern: N/A | Traces: SW-REQ-062
| Description: Initializes GPIO pin PA1 in analog mode, configures it for ADC input channel
mapping, and disables digital input buffers to reduce noise.
| Inputs: Init call, pin identifier PA1
```

```
| Outputs: Status code (OK/ERROR)
| Dependencies: MCU_HAL_GPIO, MCU_HAL_ADC, BoardConfig
| -> main.c: /* -- DD-COMP-073: ADCPinInitializer -- */
| -> main.h: /* -- DD-COMP-073: ADCPinInitializer -- */
| -> @trace DD-COMP-073 SW-REQ-062
+--

+-- DD-COMP-074 ---
| Title: ActuatorGPIOInitializer
| Type: Service | Pattern: N/A | Traces: SW-REQ-063
| Description: Configures pin PA5 as a push-pull GPIO output for actuator control and sets an
initial safe output level.
| Inputs: Init call, pin identifier PA5, initial level
| Outputs: Status code (OK/ERROR)
| Dependencies: MCU_HAL_GPIO, BoardConfig
| -> main.c: /* -- DD-COMP-074: ActuatorGPIOInitializer -- */
| -> main.h: /* -- DD-COMP-074: ActuatorGPIOInitializer -- */
| -> @trace DD-COMP-074 SW-REQ-063
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 25
================================================================================
```

27. Sub-Prompt 26 - ActuatorControlModule

Module	ActuatorControlModule
Sub-Prompt	26 of 53
Components	DD-COMP-075

```
================================================================================
SUB-PROMPT 26 OF 53
Module: ActuatorControlModule | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "ActuatorControlModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.

main.c expected:
- Implement service state machine or polling loop.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-075 ---
| Title: PWMTimerInitializer
| Type: Service | Pattern: Factory | Traces: SW-REQ-064
| Description: Initializes an advanced PWM timer with configured frequency, duty cycle resolution,
and output channel mapping to generate actuator control signals.
| Inputs: Init call, PWM frequency, duty cycle resolution, channel mapping
```

```
| Outputs: Status code (OK/ERROR), PWM timer handle
| Dependencies: MCU_HAL_TIM, BoardConfig, ActuatorGPIOInitializer
| -> main.c: /* -- DD-COMP-075: PWMTimerInitializer -- */
| -> main.h: /* -- DD-COMP-075: PWMTimerInitializer -- */
| -> @trace DD-COMP-075 SW-REQ-064
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 26
================================================================================
```

28. Sub-Prompt 27 - HAL

Module	HAL
Sub-Prompt	27 of 53
Components	DD-COMP-076, DD-COMP-079

```
================================================================================
SUB-PROMPT 27 OF 53
Module: HAL | Project: Project-3
Components: 2
================================================================================

TASK: Implement all design components for module "HAL".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.
- Define module-level enums, macros, and config constants.

main.c expected:
- Implement service state machine or polling loop.
- Implement module init, process, and deinit functions.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-076 ---
| Title: UART Peripheral Initializer
| Type: Class | Pattern: Factory | Traces: SW-REQ-065
```

```
| Description: Configures UART peripheral registers (baud rate, data bits, parity, stop bits, GPIO
muxing) and enables the UART clock and interrupts as required for external communication.
| Inputs: UART configuration structure (baud_rate, data_bits, parity, stop_bits, flow_control),
UART instance identifier
| Outputs: Initialization status (success/failure), configured UART handle
| Dependencies: ClockManager, GPIOConfigurator, InterruptController
| -> main.c: /* -- DD-COMP-076: UART Peripheral Initializer -- */
| -> main.h: /* -- DD-COMP-076: UART Peripheral Initializer -- */
| -> @trace DD-COMP-076 SW-REQ-065
+--

+-- DD-COMP-079 ---
| Title: Clock and Timer Configuration
| Type: Service | Pattern: Factory | Traces: SW-REQ-068
| Description: Configures system clock source, PLL settings, and timer prescalers to produce a 1
ms control loop period within ±1% tolerance; provides timer interrupt for control loop scheduling.
| Inputs: Clock source config, PLL multipliers/dividers, timer target period (1 ms)
| Outputs: Configured system clock and timer handles, timing accuracy status
| Dependencies: OscillatorDriver, TimerPeripheral, InterruptController
| -> main.c: /* -- DD-COMP-079: Clock and Timer Configuration -- */
| -> main.h: /* -- DD-COMP-079: Clock and Timer Configuration -- */
| -> @trace DD-COMP-079 SW-REQ-068
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 27
================================================================================
```

29. Sub-Prompt 28 - Driver

Module	Driver
Sub-Prompt	28 of 53
Components	DD-COMP-077

```
================================================================================
SUB-PROMPT 28 OF 53
Module: Driver | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "Driver".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.

main.c expected:
- Implement service state machine or polling loop.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-077 ---
| Title: ADC DMA Configuration Service
| Type: Service | Pattern: Facade | Traces: SW-REQ-066
| Description: Sets up DMA controller channels for ADC data transfers, configures transfer size,
circular/normal mode, memory buffer address, and enables DMA requests from ADC to reduce CPU load.
```

```
| Inputs: ADC handle, DMA channel config (buffer_address, length, mode, priority), sampling
trigger source
| Outputs: DMA configuration status, DMA handle linked to ADC
| Dependencies: HAL.ADCPeripheral, HAL.DMAController, BufferManager
| -> main.c: /* -- DD-COMP-077: ADC DMA Configuration Service -- */
| -> main.h: /* -- DD-COMP-077: ADC DMA Configuration Service -- */
| -> @trace DD-COMP-077 SW-REQ-066
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 28
================================================================================
```

30. Sub-Prompt 29 - Middleware

Module	Middleware
Sub-Prompt	29 of 53
Components	DD-COMP-078

```
================================================================================
SUB-PROMPT 29 OF 53
Module: Middleware | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "Middleware".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.

main.c expected:
- Implement service state machine or polling loop.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-078 ---
| Title: Independent Watchdog Service
| Type: Service | Pattern: Singleton | Traces: SW-REQ-067
| Description: Enables independent watchdog with configured timeout and provides periodic refresh
within 50% of timeout during normal operation; exposes refresh API used by application scheduler.
| Inputs: Watchdog timeout configuration, periodic refresh tick
```

```
| Outputs: Watchdog enable status, refresh success/failure
| Dependencies: HAL.WatchdogPeripheral, SystemTimer
| -> main.c: /* -- DD-COMP-078: Independent Watchdog Service -- */
| -> main.h: /* -- DD-COMP-078: Independent Watchdog Service -- */
| -> @trace DD-COMP-078 SW-REQ-067
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 29
================================================================================
```

31. Sub-Prompt 30 - Application

Module	Application
Sub-Prompt	30 of 53
Components	DD-COMP-080, DD-COMP-081, DD-COMP-082, DD-COMP-083

```
================================================================================
SUB-PROMPT 30 OF 53
Module: Application | Project: Project-3
Components: 4
================================================================================

TASK: Implement all design components for module "Application".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare all public API function prototypes.
- Declare service init and run function prototypes.
- Define module-level enums, macros, and config constants.

main.c expected:
- Implement all API handler functions as static.
- Implement service state machine or polling loop.
- Implement module init, process, and deinit functions.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-080 ---
| Title: Layered Architecture Enforcement Interface
```

```
| Type: Interface | Pattern: Adapter | Traces: SW-REQ-069
| Description: Defines abstract hardware access interfaces used by the Application layer; all
hardware interactions are routed through Middleware/Driver/HAL implementations to prevent direct
hardware access.
| Inputs: Hardware access requests (e.g., read_adc, write_uart, set_gpio)
| Outputs: Hardware operation results via interface methods
| Dependencies: Middleware.HardwareAbstractionFacade
| -> main.c: /* -- DD-COMP-080: Layered Architecture Enforcement Interface -- */
| -> main.h: /* -- DD-COMP-080: Layered Architecture Enforcement Interface -- */
| -> @trace DD-COMP-080 SW-REQ-069
+--

+-- DD-COMP-081 ---
| Title: System State Machine
| Type: Class | Pattern: State | Traces: SW-REQ-070
| Description: Implements finite state machine with states INIT, IDLE, ACTIVE, SHIPPING_LOCK, and
FAULT; processes events/commands and triggers state transitions and entry/exit actions.
| Inputs: Events (reset, init_complete, activate_cmd, shipping_lock_cmd, unlock_cmd,
fault_detected)
| Outputs: Current state, state transition notifications
| Dependencies: CommandParser, FaultManager, InitializationManager
| -> main.c: /* -- DD-COMP-081: System State Machine -- */
| -> main.h: /* -- DD-COMP-081: System State Machine -- */
| -> @trace DD-COMP-081 SW-REQ-070
+--

+-- DD-COMP-082 ---
| Title: Reset Initialization Handler
| Type: Service | Pattern: Observer | Traces: SW-REQ-071
| Description: On reset or power-up, forces state machine to INIT state and initiates
initialization sequence.
| Inputs: Reset event signal, power-up event
| Outputs: State machine set to INIT, initialization start trigger
| Dependencies: SystemStateMachine, InitializationManager
| -> main.c: /* -- DD-COMP-082: Reset Initialization Handler -- */
| -> main.h: /* -- DD-COMP-082: Reset Initialization Handler -- */
| -> @trace DD-COMP-082 SW-REQ-071
+--

+-- DD-COMP-083 ---
| Title: Nominal Transition Controller
| Type: Service | Pattern: Command | Traces: SW-REQ-072
| Description: Validates and executes nominal transitions: INIT->IDLE after initialization
success, IDLE->ACTIVE on activate command, ACTIVE->SHIPPING_LOCK on shipping lock command,
```

```
SHIPPING_LOCK->IDLE on unlock command.
| Inputs: Initialization status, command inputs (activate, shipping_lock, unlock)
| Outputs: Requested state transition, transition success/failure
| Dependencies: SystemStateMachine, CommandParser, InitializationManager
| -> main.c: /* -- DD-COMP-083: Nominal Transition Controller -- */
| -> main.h: /* -- DD-COMP-083: Nominal Transition Controller -- */
| -> @trace DD-COMP-083 SW-REQ-072
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 30
================================================================================
```

32. Sub-Prompt 31 - StateManagementModule

Module	StateManagementModule
Sub-Prompt	31 of 53
Components	DD-COMP-084

```
================================================================================
SUB-PROMPT 31 OF 53
Module: StateManagementModule | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "StateManagementModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Define module-level enums, macros, and config constants.

main.c expected:
- Implement module init, process, and deinit functions.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-084 ---
| Title: FaultStateManager
| Type: Class | Pattern: State | Traces: SW-REQ-073
| Description: Implements system state machine logic to transition to FAULT from any current state
when a fault condition is flagged, and to remain in FAULT until an explicit reset event occurs.
The manager checks fault flags each cycle and gates all non-reset transitions while in FAULT.
```

```
| Inputs: Current system state, fault_detected flag, reset_event flag
| Outputs: Next system state, state transition events
| Dependencies: FaultMonitor, ResetController, StateEventBus
| -> main.c: /* -- DD-COMP-084: FaultStateManager -- */
| -> main.h: /* -- DD-COMP-084: FaultStateManager -- */
| -> @trace DD-COMP-084 SW-REQ-073
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 31
================================================================================
```

33. Sub-Prompt 32 - WatchdogModule

Module	WatchdogModule
Sub-Prompt	32 of 53
Components	DD-COMP-085, DD-COMP-086

```
================================================================================
SUB-PROMPT 32 OF 53
Module: WatchdogModule | Project: Project-3
Components: 2
================================================================================

TASK: Implement all design components for module "WatchdogModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.

main.c expected:
- Implement service state machine or polling loop.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-085 ---
| Title: HardwareWatchdogInitializer
| Type: Service | Pattern: Factory | Traces: SW-REQ-074
| Description: Initializes the independent hardware watchdog during system startup and configures
it with the platform-supported timeout value retrieved from platform configuration or HAL.
| Inputs: Platform timeout value, startup lifecycle event
```

```
| Outputs: Configured watchdog handle, initialization status
| Dependencies: HALWatchdogDriver, PlatformConfigService
| -> main.c: /* -- DD-COMP-085: HardwareWatchdogInitializer -- */
| -> main.h: /* -- DD-COMP-085: HardwareWatchdogInitializer -- */
| -> @trace DD-COMP-085 SW-REQ-074
+--

+-- DD-COMP-086 ---
| Title: WatchdogServicer
| Type: Service | Pattern: Observer | Traces: SW-REQ-075
| Description: Services (kicks) the independent watchdog on a periodic schedule during normal
operation, ensuring at least one service within each watchdog timeout period. The service is
disabled when a fatal malfunction is declared.
| Inputs: System run state, scheduler ticks, fatal_malfunction flag
| Outputs: Watchdog service (kick) signal
| Dependencies: HALWatchdogDriver, SchedulerService, HealthStatusService
| -> main.c: /* -- DD-COMP-086: WatchdogServicer -- */
| -> main.h: /* -- DD-COMP-086: WatchdogServicer -- */
| -> @trace DD-COMP-086 SW-REQ-075
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 32
================================================================================
```

34. Sub-Prompt 33 - FaultHandlingModule

Module	FaultHandlingModule
Sub-Prompt	33 of 53
Components	DD-COMP-087

```
================================================================================
SUB-PROMPT 33 OF 53
Module: FaultHandlingModule | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "FaultHandlingModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.

main.c expected:
- Implement service state machine or polling loop.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-087 ---
| Title: FatalMalfunctionResetHandler
| Type: Service | Pattern: Command | Traces: SW-REQ-076
| Description: On detection of fatal software malfunction, inhibits WatchdogServicer and allows
watchdog timeout to occur, triggering a system reset. Logs the malfunction cause before inhibiting
servicing.
```

```
| Inputs: Fatal malfunction event, malfunction diagnostics
| Outputs: Disable-watchdog-service command, fault log entry
| Dependencies: WatchdogServicer, DiagnosticsLogger, FaultMonitor
| -> main.c: /* -- DD-COMP-087: FatalMalfunctionResetHandler -- */
| -> main.h: /* -- DD-COMP-087: FatalMalfunctionResetHandler -- */
| -> @trace DD-COMP-087 SW-REQ-076
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 33
================================================================================
```

35. Sub-Prompt 34 - SafetyControlModule

Module	SafetyControlModule
Sub-Prompt	34 of 53
Components	DD-COMP-090, DD-COMP-091

```
================================================================================
SUB-PROMPT 34 OF 53
Module: SafetyControlModule | Project: Project-3
Components: 2
================================================================================

TASK: Implement all design components for module "SafetyControlModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Define module-level enums, macros, and config constants.

main.c expected:
- Implement module init, process, and deinit functions.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-090 ---
| Title: ThresholdStopEvaluator
| Type: Class | Pattern: N/A | Traces: SW-REQ-079
| Description: Evaluates raw ADC values against configured stop thresholds each control cycle and
commands an actuator stop within the same cycle when thresholds are met or exceeded.
| Inputs: Raw ADC value, stop thresholds
```

```
| Outputs: Stop command boolean, reason code
| Dependencies: ActuatorController, ControlLoopScheduler, ThresholdConfig
| -> main.c: /* -- DD-COMP-090: ThresholdStopEvaluator -- */
| -> main.h: /* -- DD-COMP-090: ThresholdStopEvaluator -- */
| -> @trace DD-COMP-090 SW-REQ-079
+--

+-- DD-COMP-091 ---
| Title: HysteresisBandManager
| Type: Class | Pattern: State | Traces: SW-REQ-080
| Description: Applies a configurable hysteresis band around stop thresholds to prevent rapid
toggling of actuator commands. Maintains last stop state and uses upper/lower bounds for re-enable
decisions.
| Inputs: Raw ADC value, hysteresis band configuration, last stop state
| Outputs: Filtered stop command decision
| Dependencies: ThresholdStopEvaluator, ThresholdConfig
| -> main.c: /* -- DD-COMP-091: HysteresisBandManager -- */
| -> main.h: /* -- DD-COMP-091: HysteresisBandManager -- */
| -> @trace DD-COMP-091 SW-REQ-080
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 34
================================================================================
```

36. Sub-Prompt 35 - CommsModule

Module	CommsModule
Sub-Prompt	35 of 53
Components	DD-COMP-093, DD-COMP-094, DD-COMP-095, DD-COMP-096, DD-COMP-097, DD-COMP-098

```
================================================================================
SUB-PROMPT 35 OF 53
Module: CommsModule | Project: Project-3
Components: 6
================================================================================

TASK: Implement all design components for module "CommsModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare algorithm function prototypes.
- Declare service init and run function prototypes.
- Define module-level enums, macros, and config constants.

main.c expected:
- Implement algorithm logic with step-by-step inline comments.
- Implement service state machine or polling loop.
- Implement module init, process, and deinit functions.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-093 ---
```

```
| Title: UartStatusScheduler
| Type: Service | Pattern: Observer | Traces: SW-REQ-082
| Description: Schedules and triggers periodic UART status message transmission at a configurable
interval during normal operation.
| Inputs: Configured interval (ms), system tick timer, normal operation flag
| Outputs: Transmit request to UartStatusMessageBuilder
| Dependencies: SystemTimer, UartStatusMessageBuilder, UartDriver, ConfigurationStore,
SystemStateService
| -> main.c: /* -- DD-COMP-093: UartStatusScheduler -- */
| -> main.h: /* -- DD-COMP-093: UartStatusScheduler -- */
| -> @trace DD-COMP-093 SW-REQ-082
+--

+-- DD-COMP-094 ---
| Title: UartStatusMessageBuilder
| Type: Class | Pattern: Builder | Traces: SW-REQ-082
| Description: Builds the UART status payload containing raw ADC value, system state, shipping
lock status, and fault flags, and hands it to the UART driver for transmission.
| Inputs: Raw ADC counts, system state enum, shipping lock status, fault flags bitmask
| Outputs: Serialized UART frame buffer
| Dependencies: AdcDriver, SystemStateService, ShippingLockService, FaultManager, UartDriver
| -> main.c: /* -- DD-COMP-094: UartStatusMessageBuilder -- */
| -> main.h: /* -- DD-COMP-094: UartStatusMessageBuilder -- */
| -> @trace DD-COMP-094 SW-REQ-082
+--

+-- DD-COMP-095 ---
| Title: UartConfigService
| Type: Service | Pattern: Singleton | Traces: SW-REQ-083
| Description: Configures UART peripheral with required baud rate and frame format during
initialization.
| Inputs: UART peripheral handle, desired configuration parameters
| Outputs: Initialized UART peripheral state
| Dependencies: UartDriver, HardwareAbstractionLayer
| -> main.c: /* -- DD-COMP-095: UartConfigService -- */
| -> main.h: /* -- DD-COMP-095: UartConfigService -- */
| -> @trace DD-COMP-095 SW-REQ-083
+--

+-- DD-COMP-096 ---
| Title: UartBaudRateConfig
| Type: Algorithm | Pattern: N/A | Traces: SW-REQ-083
| Description: Sets UART baud rate to 115200 by computing and applying peripheral divisor values.
| Inputs: System clock frequency, target baud rate (115200)
```

```
| Outputs: UART register settings for baud rate
| Dependencies: UartDriver, HardwareAbstractionLayer
| -> main.c: /* -- DD-COMP-096: UartBaudRateConfig -- */
| -> main.h: /* -- DD-COMP-096: UartBaudRateConfig -- */
| -> @trace DD-COMP-096 SW-REQ-083
+--

+-- DD-COMP-097 ---
| Title: UartFrameFormatConfig
| Type: Algorithm | Pattern: N/A | Traces: SW-REQ-084
| Description: Configures UART frame format to 8 data bits, no parity, and 1 stop bit (8N1).
| Inputs: Frame format parameters (8N1)
| Outputs: UART register settings for frame format
| Dependencies: UartDriver, HardwareAbstractionLayer
| -> main.c: /* -- DD-COMP-097: UartFrameFormatConfig -- */
| -> main.h: /* -- DD-COMP-097: UartFrameFormatConfig -- */
| -> @trace DD-COMP-097 SW-REQ-084
+--

+-- DD-COMP-098 ---
| Title: UartTransmitIntervalValidator
| Type: Class | Pattern: N/A | Traces: SW-REQ-085
| Description: Validates configured transmit interval to be less than or equal to 100 ms and
enforces fallback to 100 ms if out of range.
| Inputs: Configured interval (ms)
| Outputs: Validated interval (ms)
| Dependencies: ConfigurationStore
| -> main.c: /* -- DD-COMP-098: UartTransmitIntervalValidator -- */
| -> main.h: /* -- DD-COMP-098: UartTransmitIntervalValidator -- */
| -> @trace DD-COMP-098 SW-REQ-085
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 35
================================================================================
```

37. Sub-Prompt 36 - FaultMonitoringModule

Module	FaultMonitoringModule
Sub-Prompt	36 of 53
Components	DD-COMP-102

```
================================================================================
SUB-PROMPT 36 OF 53
Module: FaultMonitoringModule | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "FaultMonitoringModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Define module-level enums, macros, and config constants.

main.c expected:
- Implement module init, process, and deinit functions.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-102 ---
| Title: ShortCircuitDetector
| Type: Class | Pattern: Observer | Traces: SW-REQ-089
| Description: Evaluates short-circuit indicators using either dedicated hardware flag inputs or
current measurement thresholds. On detection, it requests actuator shutdown and raises a
short-circuit fault in the fault manager.
```

```
| Inputs: current_measurement, short_circuit_flag_gpio, detection_thresholds
| Outputs: fault_event=SHORT_CIRCUIT, actuator_disable_request
| Dependencies: AdcAcquisitionService, GpioInputService, FaultManager, ActuatorControlService
| -> main.c: /* -- DD-COMP-102: ShortCircuitDetector -- */
| -> main.h: /* -- DD-COMP-102: ShortCircuitDetector -- */
| -> @trace DD-COMP-102 SW-REQ-089
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 36
================================================================================
```

38. Sub-Prompt 37 - PowerMonitoringModule

Module	PowerMonitoringModule
Sub-Prompt	37 of 53
Components	DD-COMP-103

```
================================================================================
SUB-PROMPT 37 OF 53
Module: PowerMonitoringModule | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "PowerMonitoringModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Define module-level enums, macros, and config constants.

main.c expected:
- Implement module init, process, and deinit functions.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-103 ---
| Title: UndervoltageMonitor
| Type: Class | Pattern: Observer | Traces: SW-REQ-090
| Description: Monitors supply voltage from ADC samples and triggers undervoltage fault when value
falls below 9 V. Ensures actuator output is disabled upon fault.
| Inputs: supply_voltage_sample
```

```
| Outputs: fault_event=UNDERVOLTAGE, actuator_disable_request
| Dependencies: AdcAcquisitionService, FaultManager, ActuatorControlService
| -> main.c: /* -- DD-COMP-103: UndervoltageMonitor -- */
| -> main.h: /* -- DD-COMP-103: UndervoltageMonitor -- */
| -> @trace DD-COMP-103 SW-REQ-090
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 37
================================================================================
```

39. Sub-Prompt 38 - ReliabilityModule

Module	ReliabilityModule
Sub-Prompt	38 of 53
Components	DD-COMP-104

```
================================================================================
SUB-PROMPT 38 OF 53
Module: ReliabilityModule | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "ReliabilityModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.

main.c expected:
- Implement service state machine or polling loop.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-104 ---
| Title: WatchdogService
| Type: Service | Pattern: Singleton | Traces: SW-REQ-091
| Description: Initializes and enables the hardware watchdog, periodically services it during
normal execution, and on reset records a watchdog fault flag in non-volatile fault log on next
boot.
```

```
| Inputs: system_tick, boot_reset_cause
| Outputs: watchdog_kick, fault_event=WATCHDOG_TIMEOUT
| Dependencies: HardwareWatchdogDriver, FaultLogStorage, SystemResetService
| -> main.c: /* -- DD-COMP-104: WatchdogService -- */
| -> main.h: /* -- DD-COMP-104: WatchdogService -- */
| -> @trace DD-COMP-104 SW-REQ-091
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 38
================================================================================
```

40. Sub-Prompt 39 - StateMachineModule

Module	StateMachineModule
Sub-Prompt	39 of 53
Components	DD-COMP-105

```
================================================================================
SUB-PROMPT 39 OF 53
Module: StateMachineModule | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "StateMachineModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Define module-level enums, macros, and config constants.

main.c expected:
- Implement module init, process, and deinit functions.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-105 ---
| Title: FaultStateTransitionHandler
| Type: Class | Pattern: State | Traces: SW-REQ-092
| Description: Receives fault events within the current control cycle and forces the state machine
to transition to FAULT state before cycle completion. Ensures deterministic same-cycle transition.
| Inputs: fault_event_queue, current_state
```

```
| Outputs: next_state=FAULT when any fault detected
| Dependencies: FaultManager, ControlLoopScheduler
| -> main.c: /* -- DD-COMP-105: FaultStateTransitionHandler -- */
| -> main.h: /* -- DD-COMP-105: FaultStateTransitionHandler -- */
| -> @trace DD-COMP-105 SW-REQ-092
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 39
================================================================================
```

41. Sub-Prompt 40 - SensorProcessingModule

Module	SensorProcessingModule
Sub-Prompt	40 of 53
Components	DD-COMP-106

```
================================================================================
SUB-PROMPT 40 OF 53
Module: SensorProcessingModule | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "SensorProcessingModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare algorithm function prototypes.

main.c expected:
- Implement algorithm logic with step-by-step inline comments.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-106 ---
| Title: TemperatureProcessing
| Type: Algorithm | Pattern: N/A | Traces: SW-REQ-093
| Description: Processes raw temperature sensor ADC values into engineering units using
fixed-point scaling that supports -40°C to +85°C without overflow or resolution loss. Uses
calibrated coefficients and saturates out-of-range values only after conversion.
```

```
| Inputs: raw_temperature_adc, calibration_coefficients
| Outputs: temperature_celsius_fixed_point
| Dependencies: AdcAcquisitionService, CalibrationDataStore
| -> main.c: /* -- DD-COMP-106: TemperatureProcessing -- */
| -> main.h: /* -- DD-COMP-106: TemperatureProcessing -- */
| -> @trace DD-COMP-106 SW-REQ-093
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 40
================================================================================
```

42. Sub-Prompt 41 - DataPersistenceModule

Module	DataPersistenceModule
Sub-Prompt	41 of 53
Components	DD-COMP-107

```
================================================================================
SUB-PROMPT 41 OF 53
Module: DataPersistenceModule | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "DataPersistenceModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.

main.c expected:
- Implement service state machine or polling loop.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-107 ---
| Title: NvmConfigStorage
| Type: Service | Pattern: Repository | Traces: SW-REQ-094
| Description: Manages configuration and calibration data in non-volatile memory without
software-enforced invalidation due to storage temperature range. Provides integrity checks only on
data format/version, not temperature.
```

```
| Inputs: config_write_requests, config_read_requests
| Outputs: config_data, write_status
| Dependencies: NvmDriver, DataIntegrityChecker
| -> main.c: /* -- DD-COMP-107: NvmConfigStorage -- */
| -> main.h: /* -- DD-COMP-107: NvmConfigStorage -- */
| -> @trace DD-COMP-107 SW-REQ-094
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 41
================================================================================
```

43. Sub-Prompt 42 - SystemConstraintsModule

Module	SystemConstraintsModule
Sub-Prompt	42 of 53
Components	DD-COMP-108

```
================================================================================
SUB-PROMPT 42 OF 53
Module: SystemConstraintsModule | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "SystemConstraintsModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Define module-level enums, macros, and config constants.

main.c expected:
- Implement module init, process, and deinit functions.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-108 ---
| Title: HumidityIndependencePolicy
| Type: Class | Pattern: N/A | Traces: SW-REQ-095
| Description: Defines operational policy indicating no humidity sensor dependency or
condensation-detection logic. Ensures build-time and runtime checks prevent use of humidity
inputs.
```

```
| Inputs: system_configuration
| Outputs: policy_compliance_status
| Dependencies: ConfigurationManager, BuildFeatureFlags
| -> main.c: /* -- DD-COMP-108: HumidityIndependencePolicy -- */
| -> main.h: /* -- DD-COMP-108: HumidityIndependencePolicy -- */
| -> @trace DD-COMP-108 SW-REQ-095
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 42
================================================================================
```

44. Sub-Prompt 43 - AcquisitionModule

Module	AcquisitionModule
Sub-Prompt	43 of 53
Components	DD-COMP-109

```
================================================================================
SUB-PROMPT 43 OF 53
Module: AcquisitionModule | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "AcquisitionModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.

main.c expected:
- Implement service state machine or polling loop.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-109 ---
| Title: AdcAcquisitionService
| Type: Service | Pattern: Producer-Consumer | Traces: SW-REQ-096
| Description: Continuously samples ADC channels at a fixed periodic rate using a
timer/interrupt-driven loop. Stores the latest sample per channel in a lock-free buffer accessible
by the control loop each cycle.
```

```
| Inputs: timer_tick, adc_channels_config
| Outputs: latest_adc_samples
| Dependencies: AdcDriver, TimerService, SampleBuffer
| -> main.c: /* -- DD-COMP-109: AdcAcquisitionService -- */
| -> main.h: /* -- DD-COMP-109: AdcAcquisitionService -- */
| -> @trace DD-COMP-109 SW-REQ-096
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 43
================================================================================
```

45. Sub-Prompt 44 - CommunicationModule

Module	CommunicationModule
Sub-Prompt	44 of 53
Components	DD-COMP-114

```
================================================================================
SUB-PROMPT 44 OF 53
Module: CommunicationModule | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "CommunicationModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.

main.c expected:
- Implement service state machine or polling loop.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-114 ---
| Title: UARTCommandHandler
| Type: Service | Pattern: Command | Traces: SW-REQ-099
| Description: Parses incoming UART frames and dispatches actuator ON/OFF commands. Updates
actuator enable state based on valid commands.
| Inputs: UART RX bytes, frame metadata
```

```
| Outputs: Actuator enable state updates, command acknowledgments
| Dependencies: UARTDriver, ActuatorEnableController
| -> main.c: /* -- DD-COMP-114: UARTCommandHandler -- */
| -> main.h: /* -- DD-COMP-114: UARTCommandHandler -- */
| -> @trace DD-COMP-114 SW-REQ-099
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 44
================================================================================
```

46. Sub-Prompt 45 - FaultModule

Module	FaultModule
Sub-Prompt	45 of 53
Components	DD-COMP-116, DD-COMP-117, DD-COMP-118, DD-COMP-123

```
================================================================================
SUB-PROMPT 45 OF 53
Module: FaultModule | Project: Project-3
Components: 4
================================================================================

TASK: Implement all design components for module "FaultModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare all public API function prototypes.
- Declare service init and run function prototypes.
- Define module-level enums, macros, and config constants.

main.c expected:
- Implement all API handler functions as static.
- Implement service state machine or polling loop.
- Implement module init, process, and deinit functions.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-116 ---
| Title: FaultMonitor
```

```
| Type: Service | Pattern: Observer | Traces: SW-REQ-100
| Description: Polls or receives fault input changes and detects fault conditions within 100 ms.
Emits fault events upon detection.
| Inputs: Fault input signals, timer tick
| Outputs: Fault detected events with fault code
| Dependencies: FaultInputProvider, FaultEventBus
| -> main.c: /* -- DD-COMP-116: FaultMonitor -- */
| -> main.h: /* -- DD-COMP-116: FaultMonitor -- */
| -> @trace DD-COMP-116 SW-REQ-100
+--

+-- DD-COMP-117 ---
| Title: FaultInputProvider
| Type: Interface | Pattern: Adapter | Traces: SW-REQ-100
| Description: Abstracts access to hardware or software fault inputs for the fault monitor to
evaluate.
| Inputs: Hardware fault pins, software fault flags
| Outputs: Normalized fault input states
| Dependencies: HardwareIOHAL
| -> main.c: /* -- DD-COMP-117: FaultInputProvider -- */
| -> main.h: /* -- DD-COMP-117: FaultInputProvider -- */
| -> @trace DD-COMP-117 SW-REQ-100
+--

+-- DD-COMP-118 ---
| Title: FaultLogger
| Type: Service | Pattern: Observer | Traces: SW-REQ-101
| Description: Logs detected faults to non-volatile storage with timestamp and fault code within 1
second of detection. Subscribes to fault events.
| Inputs: Fault event (timestamp, fault code)
| Outputs: Persisted log record, logging status
| Dependencies: FaultEventBus, NonVolatileStorage, RealTimeClock
| -> main.c: /* -- DD-COMP-118: FaultLogger -- */
| -> main.h: /* -- DD-COMP-118: FaultLogger -- */
| -> @trace DD-COMP-118 SW-REQ-101
+--

+-- DD-COMP-123 ---
| Title: FaultStatusTracker
| Type: Class | Pattern: Observer | Traces: SW-REQ-103
| Description: Maintains the set of active faults and provides a query to determine whether all
faults are cleared.
| Inputs: Fault detected events, fault cleared events
| Outputs: Active fault set, all-clear status
```

```
| Dependencies: FaultEventBus
| -> main.c: /* -- DD-COMP-123: FaultStatusTracker -- */
| -> main.h: /* -- DD-COMP-123: FaultStatusTracker -- */
| -> @trace DD-COMP-123 SW-REQ-103
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 45
================================================================================
```

47. Sub-Prompt 46 - DataLayer

Module	DataLayer
Sub-Prompt	46 of 53
Components	DD-COMP-119

```
================================================================================
SUB-PROMPT 46 OF 53
Module: DataLayer | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "DataLayer".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Define data structure typedefs for all data models.

main.c expected:
- Implement data access functions (read/write/validate).

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-119 ---
| Title: FaultLogStore
| Type: Database | Pattern: Repository | Traces: SW-REQ-101
| Description: Non-volatile storage schema for fault log records including timestamp and fault
code. Provides append-only write access.
| Inputs: Log record
```

```
| Outputs: Write confirmation, stored record ID
| Dependencies: NonVolatileStorageDriver
| -> main.c: /* -- DD-COMP-119: FaultLogStore -- */
| -> main.h: /* -- DD-COMP-119: FaultLogStore -- */
| -> @trace DD-COMP-119 SW-REQ-101
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 46
================================================================================
```

48. Sub-Prompt 47 - StateModule

Module	StateModule
Sub-Prompt	47 of 53
Components	DD-COMP-120, DD-COMP-121, DD-COMP-122, DD-COMP-124

```
================================================================================
SUB-PROMPT 47 OF 53
Module: StateModule | Project: Project-3
Components: 4
================================================================================

TASK: Implement all design components for module "StateModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.
- Define module-level enums, macros, and config constants.

main.c expected:
- Implement service state machine or polling loop.
- Implement module init, process, and deinit functions.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-120 ---
| Title: SystemStateMachine
| Type: Class | Pattern: State | Traces: SW-REQ-102
```

```
| Description: Controls system state transitions. Transitions from NORMAL to FAULT within 500 ms
after any fault event. Maintains current state and transition causes.
| Inputs: Fault events, recovery commands, timer tick
| Outputs: Current state, transition cause updates
| Dependencies: FaultEventBus, RecoveryCommandHandler, StateStatusProvider
| -> main.c: /* -- DD-COMP-120: SystemStateMachine -- */
| -> main.h: /* -- DD-COMP-120: SystemStateMachine -- */
| -> @trace DD-COMP-120 SW-REQ-102
+--

+-- DD-COMP-121 ---
| Title: FaultToStateTransitionService
| Type: Service | Pattern: Observer | Traces: SW-REQ-102
| Description: Listens for fault events and triggers the SystemStateMachine to transition to FAULT
within required timing constraints, providing fault cause details.
| Inputs: Fault events
| Outputs: State transition requests
| Dependencies: FaultEventBus, SystemStateMachine
| -> main.c: /* -- DD-COMP-121: FaultToStateTransitionService -- */
| -> main.h: /* -- DD-COMP-121: FaultToStateTransitionService -- */
| -> @trace DD-COMP-121 SW-REQ-102
+--

+-- DD-COMP-122 ---
| Title: RecoveryCommandHandler
| Type: Service | Pattern: Command | Traces: SW-REQ-103
| Description: Accepts recovery commands and validates that all faults are cleared before
requesting transition from FAULT to NORMAL.
| Inputs: Recovery command, current fault status
| Outputs: Recovery transition request
| Dependencies: FaultStatusTracker, SystemStateMachine
| -> main.c: /* -- DD-COMP-122: RecoveryCommandHandler -- */
| -> main.h: /* -- DD-COMP-122: RecoveryCommandHandler -- */
| -> @trace DD-COMP-122 SW-REQ-103
+--

+-- DD-COMP-124 ---
| Title: StateStatusProvider
| Type: Service | Pattern: Observer | Traces: SW-REQ-104
| Description: Exposes software-accessible status indicator for current system state and last
transition cause. Updated on each state transition.
| Inputs: State change notifications, transition cause data
| Outputs: State status structure for software access
| Dependencies: SystemStateMachine
```

```
| -> main.c: /* -- DD-COMP-124: StateStatusProvider -- */
| -> main.h: /* -- DD-COMP-124: StateStatusProvider -- */
| -> @trace DD-COMP-124 SW-REQ-104
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 47
================================================================================
```

49. Sub-Prompt 48 - InterfaceModule

Module	InterfaceModule
Sub-Prompt	48 of 53
Components	DD-COMP-125

```
================================================================================
SUB-PROMPT 48 OF 53
Module: InterfaceModule | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "InterfaceModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare all public API function prototypes.

main.c expected:
- Implement all API handler functions as static.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-125 ---
| Title: StateStatusAPI
| Type: API Endpoint | Pattern: Facade | Traces: SW-REQ-104
| Description: Provides read access to current system state and last transition cause for other
software components.
| Inputs: Status query request
```

```
| Outputs: State status structure
| Dependencies: StateStatusProvider
| -> main.c: /* -- DD-COMP-125: StateStatusAPI -- */
| -> main.h: /* -- DD-COMP-125: StateStatusAPI -- */
| -> @trace DD-COMP-125 SW-REQ-104
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 48
================================================================================
```

50. Sub-Prompt 49 - ArtifactGenerationModule

Module	ArtifactGenerationModule
Sub-Prompt	49 of 53
Components	DD-COMP-126, DD-COMP-128, DD-COMP-129

```
================================================================================
SUB-PROMPT 49 OF 53
Module: ArtifactGenerationModule | Project: Project-3
Components: 3
================================================================================

TASK: Implement all design components for module "ArtifactGenerationModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.

main.c expected:
- Implement service state machine or polling loop.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-126 ---
| Title: ArchitectureDiagramExporter
| Type: Service | Pattern: Facade | Traces: SW-REQ-105
| Description: Generates an exportable architecture diagram PDF by pulling architecture metadata
or diagram sources from the documentation repository and rendering to PDF using a diagram
renderer.
```

```
| Inputs: project_id, doc_repo_path, diagram_source_refs, export_options
| Outputs: architecture_diagram.pdf file path
| Dependencies: DocumentationRepositoryAdapter, DiagramRenderer, PdfGenerator
| -> main.c: /* -- DD-COMP-126: ArchitectureDiagramExporter -- */
| -> main.h: /* -- DD-COMP-126: ArchitectureDiagramExporter -- */
| -> @trace DD-COMP-126 SW-REQ-105
+--

+-- DD-COMP-128 ---
| Title: SchematicPdfExporter
| Type: Service | Pattern: Facade | Traces: SW-REQ-106
| Description: Exports the electrical schematic to PDF using EDA tool automation or schematic
source files and writes the result to the release artifacts directory.
| Inputs: project_id, schematic_source_path, export_options
| Outputs: schematic.pdf file path
| Dependencies: EdaToolAdapter, PdfGenerator, ArtifactStore
| -> main.c: /* -- DD-COMP-128: SchematicPdfExporter -- */
| -> main.h: /* -- DD-COMP-128: SchematicPdfExporter -- */
| -> @trace DD-COMP-128 SW-REQ-106
+--

+-- DD-COMP-129 ---
| Title: PcbGerberExporter
| Type: Service | Pattern: Facade | Traces: SW-REQ-107
| Description: Automates PCB layout export to Gerber format and stores the generated files in the
release artifacts directory.
| Inputs: project_id, pcb_layout_source_path, layer_config, export_options
| Outputs: gerber_files directory path
| Dependencies: EdaToolAdapter, ArtifactStore
| -> main.c: /* -- DD-COMP-129: PcbGerberExporter -- */
| -> main.h: /* -- DD-COMP-129: PcbGerberExporter -- */
| -> @trace DD-COMP-129 SW-REQ-107
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
==============================================================================
END SUB-PROMPT 49
```

==========

51. Sub-Prompt 50 - DocRepositoryModule

Module	DocRepositoryModule
Sub-Prompt	50 of 53
Components	DD-COMP-127

```
================================================================================
SUB-PROMPT 50 OF 53
Module: DocRepositoryModule | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "DocRepositoryModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Define module-level enums, macros, and config constants.

main.c expected:
- Implement module init, process, and deinit functions.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-127 ---
| Title: DocumentationRepositoryAdapter
| Type: Class | Pattern: Repository | Traces: SW-REQ-105
| Description: Provides access to project documentation repository contents, including diagram
sources and PDFs.
| Inputs: repo_url, credentials, file_path
```

```
| Outputs: file stream or file metadata
| Dependencies: GitClientLibrary
| -> main.c: /* -- DD-COMP-127: DocumentationRepositoryAdapter -- */
| -> main.h: /* -- DD-COMP-127: DocumentationRepositoryAdapter -- */
| -> @trace DD-COMP-127 SW-REQ-105
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 50
================================================================================
```

52. Sub-Prompt 51 - VCSModule

Module	VCSModule
Sub-Prompt	51 of 53
Components	DD-COMP-130

```
================================================================================
SUB-PROMPT 51 OF 53
Module: VCSModule | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "VCSModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.

main.c expected:
- Implement service state machine or polling loop.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-130 ---
| Title: FirmwareReleaseTagger
| Type: Service | Pattern: Command | Traces: SW-REQ-108
| Description: Ensures firmware source code exists in a version-controlled repository, creates an
annotated tag for the release, and captures a snapshot reference for delivery.
| Inputs: repo_url, release_version, tag_message, credentials
```

```
| Outputs: tag_name, commit_hash, repository_snapshot_ref
| Dependencies: GitClientLibrary
| -> main.c: /* -- DD-COMP-130: FirmwareReleaseTagger -- */
| -> main.h: /* -- DD-COMP-130: FirmwareReleaseTagger -- */
| -> @trace DD-COMP-130 SW-REQ-108
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 51
================================================================================
```

53. Sub-Prompt 52 - ReleasePackagingModule

Module	ReleasePackagingModule
Sub-Prompt	52 of 53
Components	DD-COMP-131, DD-COMP-133, DD-COMP-134

```
================================================================================
SUB-PROMPT 52 OF 53
Module: ReleasePackagingModule | Project: Project-3
Components: 3
================================================================================

TASK: Implement all design components for module "ReleasePackagingModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.

main.c expected:
- Implement service state machine or polling loop.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-131 ---
| Title: CalibrationDocIncluder
| Type: Service | Pattern: Facade | Traces: SW-REQ-109
| Description: Validates and includes the calibration procedure PDF in the release package,
copying from documentation repository or artifact store.
| Inputs: project_id, calibration_doc_path
```

```
| Outputs: release_package_entry for calibration_procedure.pdf
| Dependencies: DocumentationRepositoryAdapter, ArtifactStore, ReleasePackageBuilder
| -> main.c: /* -- DD-COMP-131: CalibrationDocIncluder -- */
| -> main.h: /* -- DD-COMP-131: CalibrationDocIncluder -- */
| -> @trace DD-COMP-131 SW-REQ-109
+--

+-- DD-COMP-133 ---
| Title: DemonstrationVideoIncluder
| Type: Service | Pattern: Facade | Traces: SW-REQ-111
| Description: Validates and includes the demonstration MP4 video file in the release package,
ensuring format and checksum.
| Inputs: project_id, video_file_path
| Outputs: release_package_entry for demo_video.mp4
| Dependencies: ArtifactStore, ReleasePackageBuilder, MediaValidator
| -> main.c: /* -- DD-COMP-133: DemonstrationVideoIncluder -- */
| -> main.h: /* -- DD-COMP-133: DemonstrationVideoIncluder -- */
| -> @trace DD-COMP-133 SW-REQ-111
+--

+-- DD-COMP-134 ---
| Title: ReleasePackageBuilder
| Type: Service | Pattern: Builder | Traces: SW-REQ-106
| Description: Assembles exported artifacts and documentation into a structured release package
directory or archive.
| Inputs: release_id, artifact_paths, package_structure_config
| Outputs: release_package_path
| Dependencies: ArtifactStore, FileSystemAdapter
| -> main.c: /* -- DD-COMP-134: ReleasePackageBuilder -- */
| -> main.h: /* -- DD-COMP-134: ReleasePackageBuilder -- */
| -> @trace DD-COMP-134 SW-REQ-106
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 52
================================================================================
```

54. Sub-Prompt 53 - TestReportingModule

Module	TestReportingModule
Sub-Prompt	53 of 53
Components	DD-COMP-132

```
================================================================================
SUB-PROMPT 53 OF 53
Module: TestReportingModule | Project: Project-3
Components: 1
================================================================================

TASK: Implement all design components for module "TestReportingModule".
All code in main.c and main.h ONLY.

HARD RULE REMINDER
------------------
ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.

main.h expected:
- Declare service init and run function prototypes.

main.c expected:
- Implement service state machine or polling loop.

COMPONENTS TO IMPLEMENT
-----------------------
+-- DD-COMP-132 ---
| Title: TestReportGenerator
| Type: Service | Pattern: Builder | Traces: SW-REQ-110
| Description: Aggregates test results for a build, formats test case summaries with pass/fail
status, and renders a PDF test report.
| Inputs: build_id, test_results_data, template_id
```

```
| Outputs: test_report.pdf file path
| Dependencies: TestResultsRepository, PdfGenerator, TemplateEngine
| -> main.c: /* -- DD-COMP-132: TestReportGenerator -- */
| -> main.h: /* -- DD-COMP-132: TestReportGenerator -- */
| -> @trace DD-COMP-132 SW-REQ-110
+--

DELIVERABLE
-----------
=== main.h ADDITIONS ===
(typedefs, macros, enums, extern declarations, function prototypes)

=== main.c ADDITIONS ===
(function implementations, static variables, logic)

Pure C code only. No explanations. No markdown.
================================================================================
END SUB-PROMPT 53
================================================================================
```